

Obsah

1	Práce s textovým souborem v php.....	2
1.1	Čtení z textového souboru pomocí ajaxu a php.....	4
2	Interaktivní validace zadávaných údajů.....	7
2.1	Interaktivní nápověda při zadávání údajů.....	7
3	Zobrazování údajů z domén třetích stran.....	9
4	Dokumenty XML.....	16
4.1	Generování dokumentu XML pomocí funkcí pro textové pole, php.....	16
4.2	Generování dokumentu XML kódem PHP – pomocí DOM.....	17
4.3	Přenos dokumentu XML ke klientovi přes XMLHttpRequest.....	20
4.3.1	Čtení tohoto XML dokumentu:.....	20
5	Interaktivní zobrazení údajů z XML katalogu.....	22
6	Generování dokumentu XML z databázové tabulky.....	28
6.1	Parametrický výběr údajů a jejich zobrazení.....	30
6.2	Zobrazení údajů z databázové tabulky na základě výběru uživatele bez opětovného načtení stránky HTML.....	31
7	Ajax a jQuery.....	33
7.1	Ajax metody.....	34
7.1.1	jQuery load () Metoda.....	34
7.1.2	Příklad.....	35
7.1.3	Callback.....	36
7.2	Metoda jQuery.ajax.....	37
7.3	jQuery - AJAX get () a sloupek () Methods.....	38
7.4	Požadavek HTTP: GET POST vs.....	38
7.4.1	jQuery.get.....	38
7.4.2	jQuery.post.....	38
7.4.3	jQuery \$.get () Metoda.....	39
7.4.4	jQuery \$.post () Metoda.....	40
7.5	jQuerygetJSON.....	43
7.6	jQuery.getScript.....	43
7.7	Odesílání formuláře.....	44
7.8	Odesílání souborů.....	44
7.9	Spolupráce s PHP.....	44
7.9.1	getScript.....	44

7.9.2	Příklad načtení HTML souboru funkcí ajax(), podobně jako na začátku funkcí load().....	44
8	Funkce automatického dokončování.....	46
9	Ovládací prvek Autocomplete.....	47
9.1.1	Vytvoření ovládacího prvku Autocomplete.....	49
10	Autocomplete textbox použití jQuery, PHP and MySQL.....	51
11	Vzdálený zdroj pro funkci automatického dokončování.....	55
12	Vývoj a ladění ajaxových aplikací.....	60
12.1	Výpis informací z hlavičky.....	60
12.1.1	Nejpoužívanější hlavičky.....	60
12.1.2	Content-Type.....	60
12.1.3	Location.....	60
12.1.4	If-Modified-Since.....	60
12.1.5	User-Agent, Server.....	60
13	Ladění ajaxových aplikací.....	62
13.1	Využití nástroje Microsoft Script Debugger.....	62
13.2	Nástroj Firebug.....	62
14	JSON : jednotný formát pro výměnu dat.....	64
14.1.1	Formát JSON.....	64
14.1.2	JSONString.....	64
14.1.3	JSONNumber.....	65
14.1.4	JSONBoolean.....	65
14.1.5	JSONNull.....	65
14.1.6	JSONArray.....	65
14.1.7	JSONObject.....	65
14.1.8	Nástroje JSONLint a JSON Editor.....	65

1 Práce s textovým souborem v php

Čtení a zápis do textových souborů

fopen – vytvoření nového souboru, případně čtení obsahu již existujícího.

syntaxe

fopen (název souboru, mode [,použit_include_path])

mode

r otevře soubor pouze pro čtení

r+ otevře soubor pro zápis i čtení a ukazatel přemístí na **začátek souboru**, soubor již musí existovat

w otevře soubor pro zápis nových dat a jeho **původní obsah zruší**. Pokud soubor daného jména dosud **neexistoval**, funkce jej **vytvoří**. Toto se používá k vytváření nových souborů.

w+ stejné jako u **w**. Soubor lze také **číst**.

a otevře soubor pro **zápis** nových dat a umístí ukazatel na **konec souboru**. Nová data budou připojena ke starým. pokud soubor daného jména neexistuje, bude vytvořen.

a+ stejné jako u **a**. Soubor lze také **číst**.

File_Exist - prověřuje, zda soubor zadaného jména existuje.

```
if (file_exist("soubor.dat"));
```

FClose – datový soubor uzavře a ukončí možnost s ním pracovat.

```
$soubor = FOpen("text.dat","r")
```

```
FClose($soubor);
```

FGetS – čte jeden řádek textu ze souboru. \$txt = FGetS("text.dat",10);

FGetC – čte jeden řádek textu ze souboru. \$txt = FGetC("text.dat");

ReadFile výpis celého souboru na standardní výstup. ReadFile("text.dat", include_path)

FPutS napíše řetězec do otevřeného souboru (stejně FWrite).

```
FPutS($jmeno_souboru,"ahoj");
```

FileSize – velikost souboru v bajtech. FileSize("text.dat")

```
echo date("d.m.y",$cas);
```

Použití fwrite

```

<?php
$fp = fopen('data.txt', 'w');
fwrite($fp, '1');
fwrite($fp, '23');
fclose($fp);

// the content of 'data.txt' is now 123 and not 23!
?>

```

1.1 Čtení z textového souboru pomocí ajaxu a php

pokus.php

```

<?php
$cislo = $_REQUEST["cislo"];
$soubor = fopen( "pokus.txt", "r" ) or die("Nemůžu otevřít soubor");
while ( ! feof( $soubor ) )
{
    $radek = fgets( $soubor, 500 );
    $pc = substr( $radek, 0,2 );
    if ($pc==$cislo)
    {
        print "$radek<br/>";
    }
}
fclose($soubor);
?>

```

soubor pokus.txt vypadá takto

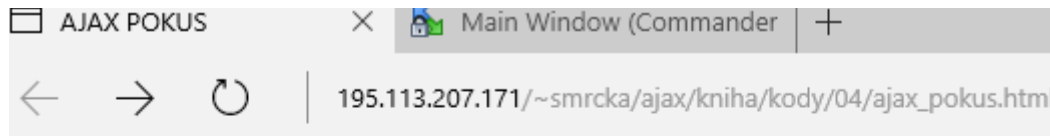
01. George Washington (1789-1797)
02. John Adams (1797-1801)
03. Thomas Jefferson (1801-1809)
04. James Madison (1809-1817)
05. James Monroe (1817-1825)
06. John Quincy Adams (1825-1829)
07. Andrew Jackson (1829-1837)
08. Martin Van Buren (1837-1841)
09. William Henry Harrison (1841)
10. John Tyler (1841-1845)
11. James Knox Polk (1845-1849)
12. Zachary Taylor (1849-1850)
13. Millard Fillmore (1850-1853)
14. Franklin Pierce (1853-1857)
15. James Buchanan (1857-1861)
16. Abraham Lincoln (1861-1865)
17. Andrew Johnson (1865-1869)

Vyzkoušení skriptu php

<http://195.113.207.163/~smrcka/wt2/pr3/ctenitxt/cteni.php?cislo=05>.

Nyní stačí vytvořit aplikaci, která bude výsledky přenášet na pozadí pomocí objektu XMLHttpRequest.

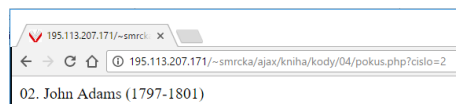
http://195.113.207.163/~smrcka/wt2/pr3/ctenitxt/ajax_pokus.html



Prezidenti USA

Poradové číslo:

Výsledek je:



Soubor ajax_pokus.html

<head>

<meta http-equiv="content-type" content="text/html;
charset=windows-1250">

<title>AJAX POKUS</title>

<script language = "javascript">

var xHttp;

function Zobraz(zdrojDat)

{

 xHttp = new XMLHttpRequest();

 xHttp.open("GET", zdrojDat);

 xHttp.onreadystatechange = function()

 {

 if (xHttp.readyState == 4 && xHttp.status == 200)

 {

 var oznam = document.getElementById("oznam");

 oznam.innerHTML = xHttp.responseText;

```
    }  
  }  
  xHttp.send(null);  
}
```

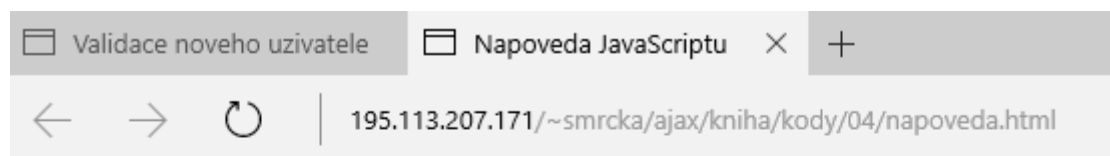
```
function DekodujParametr()  
{  
    var nn = document.getElementById("cislo");  
    var oznam = document.getElementById("oznam");  
    oznam.innerHTML = "<div></div>";  
    Zobraz("cteni.php?cislo=" + nn.value);  
}
```

```
</script>  
</head>  
<body>  
  <h2>Prezidenti USA</h2>  
  Poradove cislo:  
  <form name="potus" action="cteni.php" method="GET">  
    <input type="text" name="cislo" value="" />  
    <input type="button" value="Hledej" onclick="DekodujParametr();" />  
  </form>  
  <div id="oznam"></div>  
</body>  
</html>
```

2 Interaktivní validace zadávaných údajů

2.1 Interaktivní nápověda při zadávání údajů

Skript php bude podle části vracet pojmy. Pokud jich bude více, zobrazí je oddělené čárkou.



Interaktivni napoveda JavaScriptu

Klicove slovo JavaScriptu

break

<http://195.113.207.163/~smrcka/wt2/pr4/napoveda.html>

napoveda.php

<?php

```
$n[]="break";
$n[]="case";
$n[]="catch";
$n[]="continue";
$n[]="default";
$n[]="delete";
$n[]="do";
$n[]="else";
$n[]="false";
$n[]="finally";
$n[]="for";
$n[]="function";
$n[]="if";
$n[]="instanceof";
$n[]="new";
$n[]="switch";
$n[]="this";
$n[]="throw";
$n[]="true";
$n[]="try";
$n[]="typeof";
$n[]="return";
$n[]="var";
```

```

$n[]="void";
$n[]="while";
$n[]="with";

$key=$_GET["key"];
if (strlen($key) > 0)
{
    $hint="";
    for($i=0; $i<count($n); $i++)
    {
        if ($key == substr($n[$i],0,strlen($key)))
        {
            if ($hint=="") {$hint=$n[$i];}
            else          {$hint=$hint." , ".$n[$i];}
        }
    }
}

if ($hint == "") {$response="...";}
else {$response=$hint;}
echo $response;
?>

```

Text vrácený skriptem PHP se přenesse pomocí XMLHttpRequest a vloží do html stránky.

```

<html>
<head>
<meta http-equiv="content-type" content="text/html;
charset=windows-1250">
<title>Nápověda JavaScriptu</title>
<script language = "javascript">

var xHttp;

function Napoveda(str)
{
    if (str.length==0)
    {
        document.getElementById("napovedaDiv").innerHTML="";
        return;
    }

    var url="napoveda.php?key="+str;
    xHttp = VytvorXMLHttpRequest();
    xHttp.onreadystatechange=ZmenaStavu;
    xHttp.open("GET",url,true);
    xHttp.send(null);

```



```

}

function ZmenaStavu()
{
    if (xHttp.readyState==4 && xHttp.status == 200)
        {document.getElementById("napovedaDiv").innerHTML=xHttp.responseText;}
}

function VytvorXMLHttp()
{
    var xmlHttp= false;

    if (window.ActiveXObject)
        {xmlHttp = new ActiveXObject("Microsoft.XMLHTTP");}
    else if (window.XMLHttpRequest)
        {xmlHttp = new XMLHttpRequest();}
    return xmlHttp;
}

</script>

</head>
<body>
<h1>Interaktivní nápověda JavaScriptu</h1>
<form>
Klíčové slovo JavaScriptu <input id = "slovo" type = "text"
onkeyup = "Napoveda(this.value)">
</form>
<div id = "napovedaDiv"><div> </div></div>
</body>
</html>

```

3 Zobrazování údajů z domén třetích stran

- Z bezpečnostního hlediska není povoleno posílat dotaz na stránky z domén třetích stran.
- Když adresa URL absolutní cesty k přenášenému dokumentu směřuje do jiné domény, volání metody open vyvolá výjimku.

Řešení

Vytvořit aplikaci PHP, která zprostředkuje volání adresy URL v jiné doméně.

Data budeme brát z adresy: www.nbs.sk/KL/KLSRRRR/KLRRMMDD.HTM

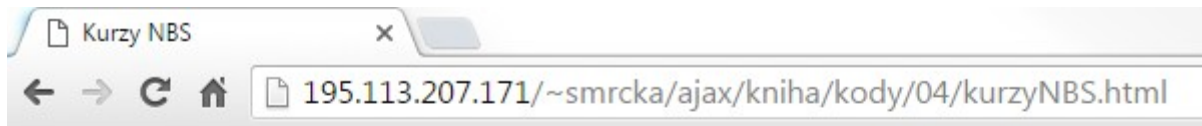
<http://www.ecb.europa.eu/stats/eurofxref/eurofxref-daily.xml>

například pro 25.2.2016 to bude

www.nbs.sk/KL/KLSL2016/KL160225.HTM

Skript pro transformaci parametrů do adresy URL, na který je kurzovní lístek.

<http://195.113.207.171/~smrcka/ajax/kniha/kody/04/kurzyNBS.html>



Kurzy NBS

Zadejte datum:

Den

Měsíc

Rok

```
<?php
//http://www.nbs.sk/KL/KLSLRRRR/KLRRMMDD.HTM
//RRRR = rok,
//RR = posledné dvojčísle roka,
//MM = mesiac,
//DD = deň.

$D = $_GET["d"];
if(strlen($D)==1) {$D ="0".$D;}
$M = $_GET["m"];
if(strlen($M)==1) {$M ="0".$M;}
$RRRR = $_GET["r"];
$R = substr($RRRR, 2,2 );

$url = "http://www.nbs.sk/KL/KLSL".$RRRR."/KL".$R.$M.$D.".HTM";

$kurzy = file_get_contents($url);
echo $kurzy;

?>
```

Poznámka

`file_get_contents` — Reads entire file into a string

Příklad

```
<?php
$homepage = file_get_contents('http://www.example.com/');
```

```
echo $homepage;  
?>
```

Pak HTML vypadá následovně

```
<head>  
<meta http-equiv="content-type" content="text/html;  
  charset=windows-1250">  
<title>Kurzy NBS</title>  
<meta http-equiv="Content-Type" content="text/html; charset=windows-1250">  
  <script language = "javascript">  
  
    var xHttp;  
  
    function Kurz( zdrojDat)  
    {  
      xHttp = VytvorXMLHttp();  
      xHttp.open("GET", zdrojDat);  
      xHttp.onreadystatechange = function()  
      {  
        if (xHttp.readyState == 4 && xHttp.status == 200)  
        {  
          var oznam = document.getElementById("oznam");  
          oznam.innerHTML = xHttp.responseText;  
        }  
      }  
      xHttp.send(null);  
    }  
  
    function DekodujParametr(udalost)  
    {  
      var d = document.getElementById("txDen");  
      var m = document.getElementById("txMesic");  
      var r = document.getElementById("txRok");  
  
      var oznam = document.getElementById("oznam");  
      oznam.innerHTML = "<div></div>";  
      Kurz("kurzyNBS.php?d=" + d.value + "&m=" + m.value + "&r=" +  
r.value);  
    }  
  
    function VytvorXMLHttp()  
    {  
      var xmlHttp= false;  
  
      if (window.ActiveXObject)  
        {xmlHttp = new ActiveXObject("Microsoft.XMLHTTP");}  
      else if (window.XMLHttpRequest)  
        {xmlHttp = new XMLHttpRequest();}  
      return xmlHttp;  
    }  
  
</script>  
  
</head>  
<body>  
  <h1>Kurzy NBS</h1>
```

```

    Zadejte datum:<br/><br/>Den
    <input type="text" id = "txDen" value = "1"
      onkeyup = "DekodujParametr(event)"/>
    <br/><br/>Měsíc
    <input type="text" id = "txMesic" value = "1"
      onkeyup = "DekodujParametr(event)"/>
    <br/><br/>Rok
    <input type="text" id = "txRok" value = "2008"
      onkeyup = "DekodujParametr(event)"/>
    <div id = "oznam"><div>
  </body>
</html>

```

Výsledek je zde:

Kurzy NBS

Zadejte datum:

Den

Měsíc

Rok

Kurzový lístok číslo 3 platný od 04.01.2008

Krajina	Mena	Množstvo	Devízy
Austrálie	AUD	1	19,983
Bulharsko	BGN	1	17,110
Kanada	CAD	1	22,809
Česká republika	CZK	1	1,279
Dánsko	DKK	1	4,400

Praktický příklad

<http://195.113.207.171/~kovari10/wt2/cv06/>

vypis tabulky

Vypiš tabulku

Kurzovní listek

29.02.2016

Měna	Jednotky	Čnb	KB	Nákup	Prodej	Nákup devize	Prodej devize	Změna
29.02.2016, 18:00								
AUD	1	17.729	17.7627	17.3186	18.2068	17.3719	18.1535	1
BGN	1	13.833	13.8331	13.5288	14.1374	1		
CAD	1	18.322	18.4078	17.9476	18.868	18.0028	18.8128	1
CHF	1	24.798	24.8821	24.1282	25.636	24.1854	25.5788	1

```
<html>
<head>
<meta http-equiv="content-type" content="text/html;
charset=windows-1250">
<title>cv06 pLTMenos z databĚ ze XML</title>
<style>
  currency, osoba {display: block; margin: 0px; border: 3px solid #73AD21;}
  date {display: block; width: 190px; height: 40px; background-color: lightgreen; align:
centermargin: auto;

  border: 3px solid #73AD21;
  padding: 10px; margin: auto;

  border: 3px solid #73AD21;
  padding: 10px;}
  name {display: inline-block; margin: 20px; width: 55px; height: 15px; }
  unit {display: inline-block; margin: 20px; width: 55px; height: 15px; }
  cnb, kb, devbuyrate, devselrate, rateprogress {display: inline-block; margin: 20px; width:
40px; height: 15px; }
  valbuyrate, valbselrate {display: inline; margin: 20px; width: 40px; height: 15px;}
  #listek, #vlozeni_listku { margin: auto;
  width: 90%;
  border: 3px solid #73AD21;
  padding: 10px; }
```

```

    jmeno, prijmeni, telefon {display: inline-block; margin:20px; width: 100px; height: 15px; }
</style>
<script type="text/javascript">
    var xHttp;

    function aktivujRequest()
    {
        xHttp = VytvorXMLHttp();
        xHttp.onreadystatechange = ZmenaStavu;
        xHttp.open("GET", "tabulka.php", true);
        xHttp.send(null);
    }
    function nactiListek(datum)
    {
        xHttp = VytvorXMLHttp();
        xHttp.onreadystatechange = ZmenaStavuL;
        xHttp.open("GET", "listek.php?datum="+datum, true);
        xHttp.send(null);
    }

    function ZmenaStavuL()
    {
        if (xHttp.readyState==4 && xHttp.status == 200)
            {document.getElementById("vlozeni_listku").innerHTML=xHttp.responseText;}
    }

    function ZmenaStavu()
    {
        if (xHttp.readyState==4 && xHttp.status == 200)
            {document.getElementById("vlozeni_obsahu").innerHTML=xHttp.responseText;}
    }

    function VytvorXMLHttp()
    {
        var xmlHttp= false;
        if (window.ActiveXObject)
            {xmlHttp = new ActiveXObject("Microsoft.XMLHTTP");}
        else if (window.XMLHttpRequest)
            {xmlHttp = new XMLHttpRequest();}
        return xmlHttp;
    }

</script>
</head>

<body>
<form>
    <h1>vypis tabulky</h1>
    <input type = "button" value = "VypiĹ tabulku"

```

```

        onclick = "aktivujRequest()">

</form>
    <div id="vlozeni_obsahu"></div>

    <br>
    <br>

    <h1>Kurzovní listek</h1>
</form>

    <input type = "date" id="datum" value = "2016-03-31"
        onchange = "nactiListek(this.value)">

</form>

<br>
    <div id="listek"><name>Měna</name><unit>Jednotky</unit><cnb>ČSN</cnb><kb>KB</
kb><valbuyrate>Nákup</valbuyrate><valbselrate>Prodej</valbselrate><devbuyrate>Nákup
devize</devbuyrate><devselrate>Prodej devize</devselrate>
<rateprogress>Změna</rateprogress>
</div>
    <div id="vlozeni_listku"></div>
</body>
</html>

```

Soubor listek.php

```

<?php
$datum = $_GET["datum"]    ;

$D =substr($datum, 8, 2);
$M =substr($datum, 5, 2);
$RRRR =substr($datum, 0, 4);

// echo $D."<br>"    ;
//echo $M."<br>"    ;
//echo $RRRR."<br>" ;

// $url = "https://www.unicreditbank.cz/web/exchange_rates_xml.php?den=".$D."".$M."".$
$RRRR;
$url = "https://www.kb.cz/kurzovni-listek/cs/rl/index.x?format=xml&filterDate=".$D."".$M."".$
$RRRR;
//    https://www.kb.cz/kurzovni-listek/cs/rl/index.x?
format=xml&filterDate=31.3.2016&webRateListId=11873

$kurzy = file_get_contents($url);

```

```
echo $kurzy;  
// echo $url;
```


4 Dokumenty XML

4.1 Generování dokumentu XML pomocí funkcí pro textové pole, php

- Pro výpis textových řetězců využijeme příkazů print a echo.
- Dokument XML vytvoříme na základě údajů uložených v datové struktuře pole v php.

XML_gen.php

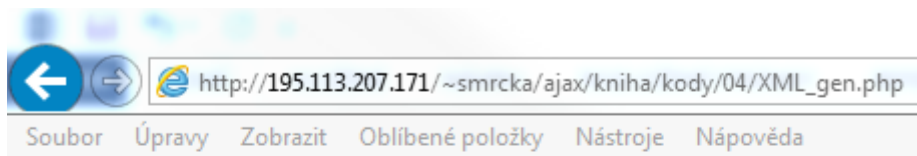
```
<?php
print '<?xml version="1.0" ?>' . "\n";
print "<filmy>\n";

$filmy=array(
    array('cislo' => '1',
        'nazev' => 'Svetaci',
        'herec' => 'Sverak',
        'rok' => '1999' ),

    array('cislo' => '2',
        'nazev' => 'Pes Filipes',
        'herec' => 'nekdo',
        'rok' => '1963' ),
    array('cislo' => '3',
        'nazev' => 'Slunce a par facek',
        'herec' => 'Troska',
        'rok' => '1999' ),
);
foreach ($filmy as $film)
{
    print " <film><br>";
    foreach($film as $znacka => $data)
    {
        print "<$znacka>".$data."</$znacka>\n";
    }
    print "</film>\n";
}
print "</filmy>\n";
?>
```

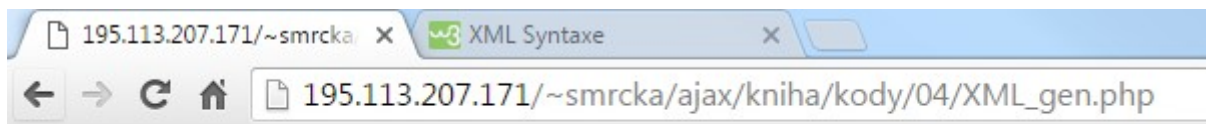
http://195.113.207.163/~smrcka/wt2/pr4/xml_gen.php

Náhled v IE



```
<?xml version="1.0"?>
- <filmy>
  - <film>
    <cislo>1</cislo>
    <nazev>Svetaci</nazev>
    <herec>Sverak</herec>
    <rok>1999</rok>
  </film>
  - <film>
    <cislo>2</cislo>
    <nazev>Pes Filipes</nazev>
    <herec>nekdo</herec>
    <rok>1963</rok>
  </film>
  - <film>
    <cislo>3</cislo>
    <nazev>Slunce a par facek</nazev>
    <herec>Troska</herec>
    <rok>1999</rok>
  </film>
</filmy>
```

Náhled v Google Chrome



1 Svetaci Sverak 1999 2 Pes Filipes nekdo 1963 3 Slunce a par facek Troska 1999

4.2 Generování dokumentu XML kódem PHP – pomocí DOM

- Je mnohem univerzálnější

http://195.113.207.163/~smrcka/wt2/pr4/xml_dom.php

XML_dom.php

```
<?php
    $dom = new DOMDocument();

    //root element <filmy>
    // Create new element node
    $filmy = $dom->createElement('filmy');
```

```

$dom->appendChild($filmy);

    // položky elementu <film>
    $cislo = $dom->createElement('cislo');
    //vytvoří nový textový uzel (vloží text do elementu)
    $t_cislo = $dom->createTextNode('1');
    $cislo->appendChild($t_cislo);

    $nazev = $dom->createElement('nazev');
    $t_nazev = $dom->createTextNode('Dr. No');
    $nazev->appendChild($t_nazev);
    $herec = $dom->createElement('herec');
    $t_herec = $dom->createTextNode('Connery');
    $herec->appendChild($t_herec);
    $rok = $dom->createElement('nazev');
    $t_rok = $dom->createTextNode('1962');
    $rok->appendChild($t_rok);

    //element <film>
    $film = $dom->createElement('film');
    //Přidává nového potomka na konec
    $filmy->appendChild($film);
    $film->appendChild($cislo);
    $film->appendChild($nazev);
    $film->appendChild($herec);
    $film->appendChild($rok);

    //vnoreni elementu <film> pod <filmy>
    $filmy->appendChild($film);

    //XML ako string
    $sXML = $dom->saveXML();
    echo $sXML;
?>

```

Poznámka DOM document v PHP

<http://php.net/manual/en/domdocument.createelement.php>

- [DOMNode :: appendChild \(\)](#) - Přidává nového potomka na konec
- [DOMDocument :: createAttribute \(\)](#) - Vytvořit nový atribut
- [DOMDocument :: createAttributeNS \(\)](#) - Vytvořit nový atribut uzel s příslušným názem
- [DOMDocument :: createCDATASection \(\)](#) - Vytvořit nový CDATA uzel
- [DOMDocument :: createComment \(\)](#) - Vytvořit nový komentář uzel
- [DOMDocument :: createDocumentFragment \(\)](#) - Vytvořit nový dokument fragment
- [DOMDocument :: createElementNS \(\)](#) – vytvoří nový prvek uzel s příslušným názvů

Daší informace: <http://php.net/manual/en/domdocument.createelement.php>

- [DOMDocument :: createEntityReference \(\)](#) - vytvořit nový referenční entita uzel
- [DOMDocument :: createProcessingInstruction \(\)](#) - vytvoří nový uzel
- [DOMDocument :: createTextNode \(\)](#) - Vytvořit nový textový uzel

http://195.113.207.171/~smrcka/ajax/kniha/kody/04/XML_dom.php

Výsledek je



The screenshot shows a web browser window with the address bar displaying `http://195.113.207.171/~smrcka/ajax/kniha/kody/04/XML_dom.php`. The browser's menu bar includes 'Soubor', 'Úpravy', 'Zobrazit', 'Oblíbené položky', 'Nástroje', and 'Nápověda'. Below the menu bar, there is a 'Favorites' section with a star icon and the text 'Oblíbené položky', followed by a search bar containing the same URL. The main content area of the browser displays an XML document with the following structure:

```
<?xml version="1.0" ?>
- <filmy>
- <film>
  <cislo>1</cislo>
  <nazev>Dr. No</nazev>
  <herec>Connery</herec>
  <nazev>1962</nazev>
</film>
</filmy>
```

4.3 Přenos dokumentu XML ke klientovi přes XMLHttpRequest

Vytvoříme skript, který bude volat XML_dom.php.

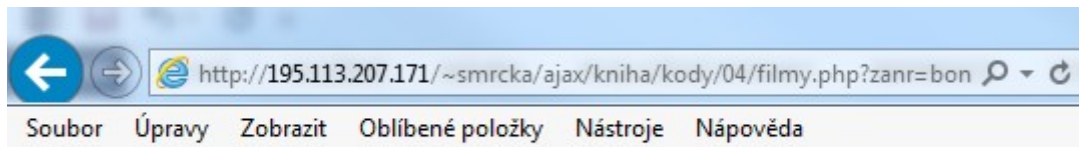
Na základě parametru v URL nám skript vygeneruje seznam filmů

Vyzkoušíme:

<http://195.113.207.171/~smrcka/ajax/kniha/kody/04/filmy.php?zanr=bondovky>

```
<?php
if ($_REQUEST["zanr"] == "bondovky")
    $nazvy = array('Dr. No', 'From Russia With Love', 'Goldfinger');
if ($_REQUEST["zanr"] == "komedie")
    $nazvy = array('Beethoven', 'Postrach Denis', 'Na strome');
echo '<?xml version="1.0"?>';
echo '<filmy>';
foreach ($nazvy as $text)
{
    echo '<film>';
    echo $text;
    echo '</film>';
}
echo '</filmy>';

?>
```

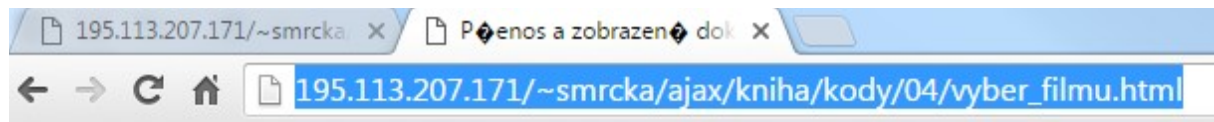


```
<?xml version="1.0"?>
- <filmy>
    <film>Dr. No</film>
    <film>From Russia With Love</film>
    <film>Goldfinger</film>
</filmy>
```

4.3.1 Čtení tohoto XML dokumentu:

K převzetí údajů stačí responseText.

http://195.113.207.163/~smrcka/wt2/pr4/vyber_filmu.html



Výběr filmu podle žánru

Dr. NoFrom Russia With LoveGoldfinger

```
<html>
<head>
<meta http-equiv="content-type" content="text/html;
  charset=windows-1250">
<title>Přenos a zobrazení dokumentu XML</title>

<script type="text/javascript">
  var xHttp;

  function aktivujRequest(pZanr)
  {
    xHttp = VytvorXMLHttp();
    xHttp.onreadystatechange = ZmenaStavu;
    xHttp.open("GET", "filmy.php?zanr="+pZanr, true);
    xHttp.send(null);
  }

  function ZmenaStavu()
  {
    if (xHttp.readyState==4 && xHttp.status == 200)
      {document.getElementById("vlozeni_obsahu").innerHTML=xHttp.responseText;}
  }

  function VytvorXMLHttp()
  {
    var xmlHttp= false;
    if (window.ActiveXObject)
      {xmlHttp = new ActiveXObject("Microsoft.XMLHTTP");}
    else if (window.XMLHttpRequest)
      {xmlHttp = new XMLHttpRequest();}
    return xmlHttp;
  }

</script>
</head>

<body>
<form>
  <h1>Výběr filmu podle žánru</h1>
  <input type = "button" value = "Komedie"
    onclick = "aktivujRequest('komedie')">
  <input type = "button" value = "Bondovky"
    onclick = "aktivujRequest('bondovky')">
</form>
  <div id="vlozeni_obsahu"></div>
</body>
</html>
```

5 Interaktivní zobrazení údajů z XML katalogu

K zobrazení uzlů dokumentu XML můžeme použít funkce objektového modelu DOM PHP.

Na adrese <http://php.net/manual/en/class.domdocument.php> je jejich popis

[DOMDocument :: konstrukt](#) - Vytvoří nový objekt DOMDocument

[DOMDocument :: createAttribute](#) - Vytvořit nový atribut

[DOMDocument :: createAttributeNS](#) - Vytvořit nový atribut uzel s příslušným názvem

[DOMDocument :: createCDATASection](#) - Vytvořit nový CDATA uzel

[DOMDocument :: createComment](#) - Vytvořit nový komentář uzel

[DOMDocument :: createDocumentFragment](#) - Vytvořit nový dokument fragment

[DOMDocument :: createElement](#) - Vytvořit nový prvek uzel

[DOMDocument :: createElementNS](#) - Vytvořit nový prvek uzel s příslušným názvem

[DOMDocument :: createEntityReference](#) - Vytvořit nový referenční entita uzel

[DOMDocument :: createProcessingInstruction](#) - vytvoří nový uzel PI

[DOMDocument :: createTextNode](#) - Vytvořit nový textový uzel

[DOMDocument :: getElementById](#) - Hledá prvek s určitým id

[DOMDocument :: getElementsByTagName](#) - Vyhledá všechny prvky s danou místní
název značky

[DOMDocument :: getElementsByTagNameNS](#) - Vyhledá všechny prvky s daným
názevem značky v určeném prostoru jmen

[DOMDocument :: importNode](#) - Import uzel do aktuálního dokumentu

[DOMDocument :: zatížení](#) - Zatížení XML ze souboru

[DOMDocument :: loadHTML](#) - Load HTML z řetězce

[DOMDocument :: loadHTMLFile](#) - Load HTML ze souboru

[DOMDocument :: loadXML](#) - Load XML z řetězce

[DOMDocument :: normalizeDocument](#) - normalizuje dokument

[DOMDocument :: registerNodeClass](#) - Registrace rozšířené třídu použitou k vytvoření
základní typ uzlu

[DOMDocument :: relaxNGValidate](#) - provádí ověření RelaxNG na dokumentu

[DOMDocument :: relaxNGValidateSource](#) - provádí ověření RelaxNG na dokumentu

[DOMDocument :: uložit](#) - vypíše vnitřní stromu XML zpět do souboru

[DOMDocument :: saveHTML](#) - vypíše vnitřní dokument do řetězce pomocí HTML
formátování

[DOMDocument :: saveHTMLFile](#) - Uloží interního dokumentu do souboru pomocí HTML
formátování

[DOMDocument :: saveXML](#) - vypíše vnitřní stromu XML zpět do řetězce

[DOMDocument :: schemaValidate](#) - Ověřuje dokument založený na schématu

[DOMDocument :: schemaValidateSource](#) - Ověřuje dokument založený na schématu

[DOMDocument :: validate](#) - Ověřuje dokument na základě jeho DTD

[DOMDocument :: XInclude](#) - Náhražky XIncludes v DOMDocument objektu

Důležité funkce

domDocument->loadxml(string document) – tato funkce načte textový řetězec obsahující document XML. V našem případě by se tato funkce volala s parametrem vráceným funkcí **file_get_contents**, která načte obsah souboru do textového řetězce.

bool DomNode-> has_child_nodes(void)

ověří, zda daný uzel obsahuje potomky. Pokud ano, můžeme použít funkci, která slouží k získání prvního potomka dotyčného uzlu.

object DomNode-> first_child(void)

Sourozence zjistíme pomocí funkce:

object DomNode-> next_sibling(void)

Tato funkce vrací při nenalezení dalších záznamů false a u novějších PHP NULL.

Příklad

Zobrazení údajů z katalogového souboru XML.

Katalog:

katalog.xml

```
<?xml version="1.0" encoding="windows-1250"?>
<SHOP>
  <SHOPITEM>
    <PRODUCT>Mořský svět</PRODUCT>
    <DESCRIPTION>Záběry života mořských živočichů</DESCRIPTION>
    <URL>http://vasobchod.cz/filmoteka/morsky-svet</URL>
    <IMGURL>http://vasobchod.cz/obrazky/filmoteka/morsky-svet.jpg</IMGURL>
    <PRICE>990</PRICE>
    <VAT>0.19</VAT>
    <PRICE_VAT>1178</PRICE_VAT>
  </SHOPITEM>
  <SHOPITEM>
    <PRODUCT>Suchozemci</PRODUCT>
    <DESCRIPTION>Suchozemští živočichové, jak je neznáte</DESCRIPTION>
    <URL>http://vasobchod.cz/filmoteka/suchozemci</URL>
    <IMGURL>http://vasobchod.cz/obrazky/filmoteka/suchozemci.jpg</IMGURL>
    <PRICE>1895</PRICE>
    <VAT>0.19</VAT>
    <PRICE_VAT>2255</PRICE_VAT>
  </SHOPITEM>
</SHOP>
```


</SHOP>

Zpracování pomocí PHP

Ten vybere údaje na úrovni elementu PRODUCT. Tyto elementy jsou na hierarchické úrovni `nodeType=1`

```
<?php
    $q=$_GET["key"];
    $xmlDoc = new DOMDocument();
    $xmlDoc->load("katalog.xml");
    $x=$xmlDoc->getElementsByTagName('PRODUCT'); //obsahuje všechny elementy
v tagu //PRODUCT

    for ($i=0; $i<=$x->length-1; $i++) //do počtu elementů
    {
        if ($x->item($i)->nodeType==1) // nodeType
//Získá typ uzlu. Jeden z předdefinovaných, 1 ... uzel je element

        {
            if ($x->item($i)->childNodes->item(0)->nodeValue == $q)
            {
                $y=($x->item($i)->parentNode);
            }
        }
    }

    $zb=($y->childNodes);

    for ($i=0;$i<$zb->length;$i++)
    {
        if ($zb->item($i)->nodeType==1)
        {
            echo($zb->item($i)->nodeName);
            echo(": ");
            echo($zb->item($i)->childNodes->item(0)->nodeValue);
            echo("<br />");
        }
    }
?>
```

Nápověda k příkazům PHP

<http://php.net/manual/en/class.domnode.php#domnode.props.nodeType>

<http://php.net/manual/en/dom.constants.php>

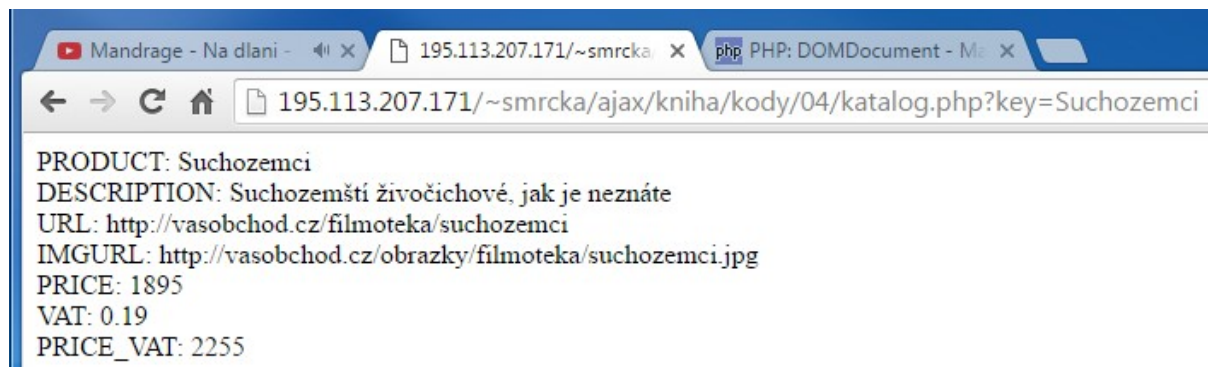
Konstantní	Hodnota	Popis
<code>XML_ELEMENT_NODE</code> (integer)	1	Uzel je DOMElement
<code>XML_ATTRIBUTE_NODE</code> (integer)	2	Uzel je DOMAttribute
<code>XML_TEXT_NODE</code> (integer)	3	Uzel je DOMText
<code>XML_CDATA_SECTION_NODE</code> (integer)	4	Uzel je DOMCdataSection
<code>XML_ENTITY_REF_NODE</code> (integer)	5	Uzel je DOMEntityReference

Konstantní	Hodnota	Popis
<code>XML_ENTITY_NODE</code> (integer)	6	Uzel je DOMEntity
<code>XML_PI_NODE</code> (integer)	7	Uzel je DOMProcessingInstruction
<code>XML_COMMENT_NODE</code> (integer)	8	Uzel je DOMComment
<code>XML_DOCUMENT_NODE</code> (integer)	9	Uzel je DOMDocument
<code>XML_DOCUMENT_TYPE_NODE</code> (integer)	10	Uzel je DOMDocumentType
<code>XML_DOCUMENT_FRAG_NODE</code> (integer)	11	Uzel je DOMDocumentFragment
<code>XML_NOTATION_NODE</code> (integer)	12	Uzel je DOMNotation
<code>XML_HTML_DOCUMENT_NODE</code> (integer)	13	

Kod vyzkoušíme:

<http://195.113.207.163/~smrcka/wt2/pr4/katalog.php?key=Suchozemci>

Výsledek



Stránka HTML bude klasika.

<http://195.113.207.163/~smrcka/wt2/pr4/katalog.html>

Mandrage - Na dlani XML katalog PHP: DOMDocument - Me

195.113.207.171/~smrcka/ajax/kniha/kody/04/katalog.html

Interaktivní katalog

Vyber produkt: Suchozemci

PRODUCT: Suchozemci
 DESCRIPTION: Suchozemští živočichové, jak je neznáte
 URL: http://vasobchod.cz/filmoteka/suchozemci
 IMGURL: http://vasobchod.cz/obrazky/filmoteka/suchozemci.jpg
 PRICE: 1895
 VAT: 0.19
 PRICE_VAT: 2255

A zde je zdroj:

```
<html>
<head>
<meta http-equiv="content-type" content="text/html;
  charset=windows-1250">
<title>XML katalog</title>
  <script language = "javascript">

    var xHttp= false;

    if (window.ActiveXObject)
      {xHttp = new ActiveXObject("Microsoft.XMLHTTP");}
    else if (window.XMLHttpRequest)
      {xHttp = new XMLHttpRequest();}

    function ZobrazDetaily(str)
    {
      var url="katalog.php" +"?key="+str;
      xHttp.onreadystatechange=ZmenaStavu;
      xHttp.open("GET",url,true);
      xHttp.send(null);
    }

    function ZmenaStavu()
    {
      if (xHttp.readyState==4 && xHttp.status == 200)
        {document.getElementById("napovedaDiv").innerHTML=xHttp.responseText;}
    }

  </script>
</head>

<body>
<h1>Interaktivní katalog</h1>
<form>Vyber produkt:
  <select name="selProdukt" onchange="ZobrazDetaily(this.value)">
    <option value="Mořský svět">Mořský svět</option>
    <option value="Suchozemci">Suchozemci</option>
  </select>
</form><p>
<div id="napovedaDiv"><b>Informace o produktu.</b></div>
```

</p></body>
</html>

6 Generování dokumentu XML z databázové tabulky

Vytvoříme v mysql tabulku

```
CREATE TABLE zakaznici
(
    id_zak int NOT NULL,
    firma varchar(20) NOT NULL,
    kontakt_jmeno varchar(20),
    adresa varchar(25),
    mesto varchar(15),
    obrat decimal(9,2),
    dluh decimal(9,2)

);

INSERT INTO zakaznici
VALUES(1, 'Procesory s.r.o', 'Vonasek Jiri', 'Volkrova 12', 'Praha', 2567892.30,
365000);

INSERT INTO zakaznici
VALUES (2,'Matice a.s', 'Kucera Jan', 'Zaoralova 16', 'Brno', 753275.65, 100000);

INSERT INTO zakaznici
VALUES (3,'Kosmetika', 'Chocholousek Libor', 'Dolni 654', 'Uhersky Brod',
212356.20, 0);

INSERT INTO zakaznici
VALUES (4,'Vodovahy a.s', 'Krupickova Alena', 'Zelena 4', 'Rajhrad', 6256120.45,
40000);

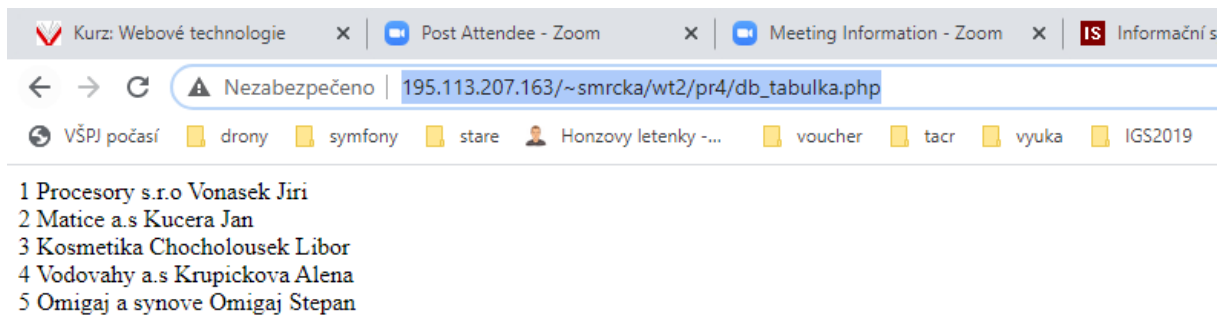
INSERT INTO zakaznici
VALUES (5,'Omigaj a synove', 'Omigaj Stepan', 'Nadrazni 333', 'Plzen', 57210,
920000);
```

Zobrazení dat

http://195.113.207.163/~smrcka/wt2/pr4/db_tabulka.php

```
<?php
$mysql = MySQL_Connect("localhost","smrcka","heslo");
$spojeni=MySQL_Select_DB("smrcka");
$result = mysql_query("SELECT id_zak, firma, kontakt_jmeno FROM zakaznici");

while ($row = mysql_fetch_object($result))
{
    echo $row->id_zak;
    echo " , ";
    echo $row->firma;
    echo " , ";
    echo $row->kontakt_jmeno;
    echo "<br>";
}
mysql_free_result($result);
?>
```



Soubor pro vygenerování XML z databáze

http://195.113.207.163/~smrcka/wt2/pr4/XML_db.php

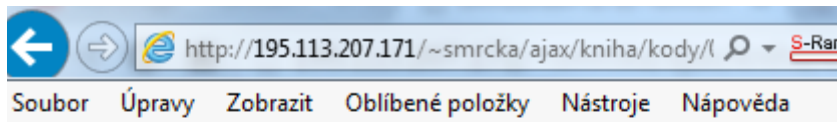
```
<?php
    print '<?xml version="1.0"?>' . "\n";
    print "<zakaznici>";

    $mysql = MySQL_Connect("localhost", "smrcka", "heslo");
    $spojeni=MySQL_Select_DB("smrcka");

    @$result = mysql_query("SELECT id_zak, firma, kontakt_jmeno FROM zakaznici ");
    while ($row = mysql_fetch_array($result))
    {
        print " <zakaznik>";
        print "     <id>".htmlspecialchars($row["id_zak"])."</id>";
        print "     <firma>".htmlspecialchars($row["firma"])."</firma>";
        print "     <kontakt_jmeno>".htmlspecialchars($row["kontakt_jmeno
    "])."</kontakt_jmeno>";
        print " </zakaznik>";
    }
    mysql_free_result($result);

    mysql_close($mysql);
    print "</zakaznici>";
?>
```

Výsledek



```
<?xml version="1.0"?>
- <zakaznici>
  - <zakaznik>
    <id>1</id>
    <firma>Procesory s.r.o</firma>
    <kontakt_jmeno/>
  </zakaznik>
  - <zakaznik>
    <id>2</id>
    <firma>Matice a.s</firma>
    <kontakt_jmeno/>
  </zakaznik>
  - <zakaznik>
    <id>3</id>
    <firma>Kosmetika</firma>
    <kontakt_jmeno/>
  </zakaznik>
  - <zakaznik>
    <id>4</id>
    <firma>Vodovahy a.s</firma>
    <kontakt_jmeno/>
  </zakaznik>
  - <zakaznik>
    <id>5</id>
    <firma>Omigaj a synove</firma>
    <kontakt_jmeno/>
  </zakaznik>
</zakaznici>
```

6.1 Parametrický výběr údajů a jejich zobrazení

Chceme přidat where

http://195.113.207.163/~smrcka/wt2/pr4/db_tabulka2.php?id=3

db_tabulka2.php

```
<html><body>
<table border=1 cellpadding=2>
<tr>
  <th>ID</th>
  <th>Firma</th>
  <th>Kontakt</th>
</tr>
```

```
<?php
$id=$_GET["id"];

mysql_connect("localhost");
mysql_select_db("test");
```

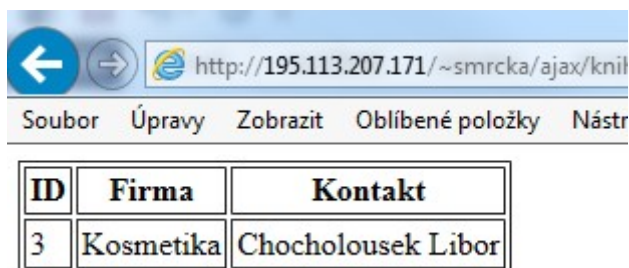
```

    $sql="SELECT id_zak, firma, kontakt_jmeno FROM zakaznici WHERE id_zak = '$id.'";
    $vys = mysql_query($sql);

    while($zaznam=mysql_fetch_array($vys))
    {
        echo "<tr>";
        echo "<td>".$zaznam["id_zak"]."</td>";
        echo "<td>".$zaznam["firma"]."</td>";
        echo "<td>".$zaznam["kontakt_jmeno"]."</td>";
        echo "</tr>";
    }
    echo "</table>";
    mysql_close();
?>
</body>
</html>

```

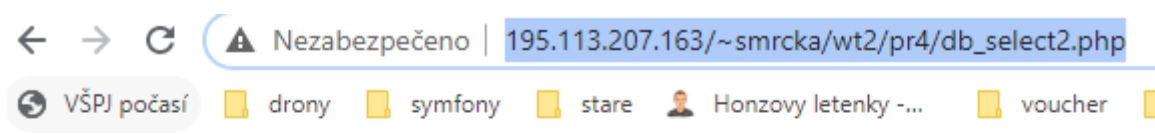
Výsledek je:



ID	Firma	Kontakt
3	Kosmetika	Chochołousek Libor

6.2 Zobrazení údajů z databázové tabulky na základě výběru uživatele bez opětovného načtení stránky HTML

http://195.113.207.163/~smrcka/wt2/pr4/db_select2.php



Interaktivní databazový katalog

Vyber firmu:

3 Kosmetika Chochołousek Libor

```

<html>
<head>
<meta http-equiv="content-type" content="text/html;
  charset=windows-1250">
<title>Výběr údajů</title>

```



```

</head>
<script language = "javascript">

    var xhttp = false;

    if (window.ActiveXObject)
        {xhttp = new ActiveXObject("Microsoft.XMLHTTP");}
    else if (window.XMLHttpRequest)
        {xhttp = new XMLHttpRequest();}

    function ZobrazDetaily(str)
    {
        var url="db_tabulka.php" +"?id="+str;
        xhttp.onreadystatechange=ZmenaStavu;
        xhttp.open("GET",url,true);
        xhttp.send(null);
    }

    function ZmenaStavu()
    {
        if (xhttp.readyState==4 && xhttp.status == 200)
            {document.getElementById("napovedaDiv").innerHTML=xhttp.responseText;}
    }
</script>

<body>
<h1>Interaktivní databazový katalog</h1>
Vyber firmu:
<select name="users" onchange="ZobrazDetaily(this.value)">

<?php

    mysql_connect("localhost");
    mysql_select_db("test");
    $sql="SELECT id_zak, firma, kontakt_jmeno FROM zakaznici";
    $vys = mysql_query($sql);

    while($zaznam=mysql_fetch_array($vys))
    {
        echo "<option value="."$zaznam["id_zak"].">".$zaznam["firma"]."</option>";
    }
    mysql_close();
?>
</select>
<p><div id="napovedaDiv"><b>Informace o firmě.</b></div></p>

</body>
</html>

```

7 Ajax a jQuery

<http://jquery-navod.cz/kategorie-ajax/9-ajax>

jQuery nabízí několik způsobů pro funkčnost AJAX.

S metodami jQuery Ajax, lze požádat o text, HTML, XML, nebo JSON ze vzdáleného serveru pomocí HTTP GET a HTTP POST

Lze načíst externí data přímo do vybraných prvků HTML webové stránky!

Bez jQuery je kódování složitější!

- různé prohlížeče mají různé syntaxe pro implementaci AJAX.
- jQuery má vše ošetřeno, takže lze psát funkčnost AJAX pouze jediný řádek kódu.

Příklad

Načti externí obsah

https://www.w3schools.com/jquery/tryit.asp?filename=tryjquery_ajax_load

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.1.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("#div1").load("demo_test.txt");
    });
});
</script>
</head>
<body>

<div id="div1"><h2>Let jQuery AJAX Change This Text</h2></div>

<button>Get External Content</button>

</body>
</html>
```

jQuery and AJAX is FUN!

This is some text in a paragraph.

Get External Content

7.1 Ajax metody

7.1.1 jQuery load () Metoda

S její pomocí načteme do vybraného prvku/vybraných prvků obsah souboru ze serveru.

Syntax:

```
$(selector).load(URL,data,callback);
```

Povinný parametr URL specifikuje URL, kterou chcete načíst.

Volitelný parametr dat specifikuje množinu QueryString párů klíč / hodnota poslat spolu s žádostí.

Volitelný parametr pro zpětné volání je název funkce, které mají být provedeny po dokončení metoda load ().

Zde je obsah náš příklad souboru: "demo_test.txt":

```
<h2>jQuery and AJAX is FUN!!!</h2>
<p id="p1">This is some text in a paragraph.</p>
```

Následující příklad načte obsah souboru "demo_test.txt" do určitého <div> prvek:

```
$("#div1").load("demo_test.txt");
```

Je také možné přidat volič jQuery parametru URL.

Následující příklad načte obsah prvku s id = "P1", uvnitř souboru "demo_test.txt", do specifického <div> prvek:

https://www.w3schools.com/jquery/tryit.asp?filename=tryjquery_ajax_load

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.1.1/jquery
.min.js"></script>
```

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("#div1").load("demo_test.txt");
    });
});
</script>
</head>
<body>
```

```
<div id="div1"><h2>Let jQuery AJAX Change This Text</h2></div>
```

```
<button>Get External Content</button>
```

```
</body>
</html>
```

Let jQuery AJAX Change This Text

Get External Content

jQuery and AJAX is FUN!

This is some text in a paragraph.

Get External Content

7.1.2 Příklad

https://www.w3schools.com/jquery/tryit.asp?filename=tryjquery_ajax_load2

Následující příklad načte obsah prvku s id = "p1", uvnitř souboru "demo_test.txt", do prvku <div>

```
$("#div1").load("demo_test.txt #p1");
```

Let jQuery AJAX Change This Text

Get External Content

This is some text in a paragraph.

Get External Content

```
<!DOCTYPE html>
<html>
<head>
```

```

<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.1.1/jquery
.min.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("#div1").load("demo_test.txt #p1");
    });
});
</script>
</head>
<body>

<div id="div1"><h2>Let jQuery AJAX Change This Text</h2></div>

<button>Get External Content</button>

</body>
</html>

```

7.1.3 Callback

`$(selector).load(URL,data,callback);`

Volitelný parametr callback určuje funkci zpětného volání spustit, když je dokončen metoda load (). Funkce zpětného volání může mít různé parametry:

- responseTxt - obsahuje výsledný obsah
- statusTxt - obsahuje stav
- XHR - obsahuje objekt XMLHttpRequest

Následující příklad zobrazí varovné pole poté, co se metoda load() dokončí.

Zobrazí se "Externí obsah úspěšně načten!", A pokud to selže, zobrazí chybové hlášení:

Příklad

```

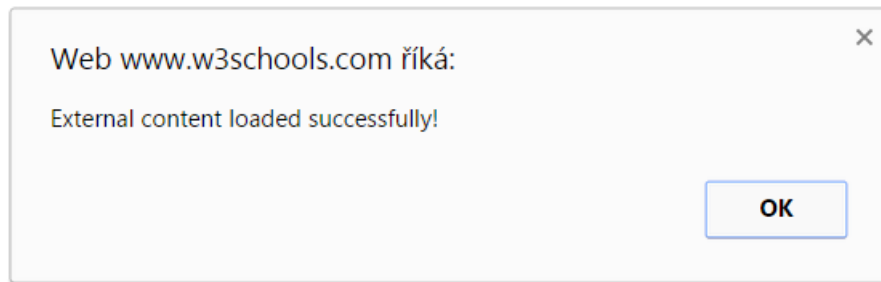
$("button").click(function(){
    $("#div1").load("demo_test.txt", function(responseTxt, statusTxt, xhr){
        if(statusTxt == "success")
            alert("External content loaded successfully!");
        if(statusTxt == "error")
            alert("Error: " + xhr.status + ": " + xhr.statusText);
    });
});

```

https://www.w3schools.com/jquery/tryit.asp?filename=tryjquery_ajax_load_callback

Let jQuery AJAX Change This Text

Get External Content



Celý příklad

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.1.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $("#div1").load("demo_test.txt", function(responseTxt, statusTxt, xhr){
      if(statusTxt == "success")
        alert("External content loaded successfully!");
      if(statusTxt == "error")
        alert("Error: " + xhr.status + ": " + xhr.statusText);
    });
  });
});
</script>
</head>
<body>

<div id="div1"><h2>Let jQuery AJAX Change This Text</h2></div>

<button>Get External Content</button>
</body>
</html>
```

7.2 Metoda jQuery.ajax

Toto je základní metoda, a všechny následující metody jsou jenom obálky pro metodu jQuery.ajax. Tato metoda má jen jeden vstupní parametr - objekt obsahující všechna nastavení, dál jsou parametry, které je vhodné pamatovat:

- **async** - požadavky jsou asynchronní, výchozí hodnota true
- **cache** - zap./vyp. caching dat prohlížeči, výchozí hodnota true

- **contentType** - výchozí nastavení "application/x-www-form-urlencoded"
- **data** - odesílaná data (řádek nebo objekt)
- **dataFilter** - filtr vstupních dat
- **dataType** - typ dat vracejících se do callback funkce (xml, html, script, json, text, _default)
- **global** - je odpovědný za použití globálních Ajax Eventů, výchozí nastavení true
- **ifModified** - kontroluje zda nastala nějaká změna v odpovědi serveru, aby neposílal další požadavek, výchozí nastavení false
- **jsonp** - přepnout jméno callback funkce pro práci s JSONP
- **processData** - vytváří objekt z odesílaných dat a posílá jako "application/x-www-form-urlencoded", jestli nutné, jinak zakazuje
- **scriptCharset** - kódování - aktuální pro JSONP
- **timeout** - časový limit v ms
- **type** - GET nebo POST
- **url** - url požadované stránky

Lokální AJAX Eventy:

- **beforeSend** - zapína se před odesláním požadavku.
- **error** - jestli nastala chyba
- **success** - jestli chyby nejsou
- **complete** - zapína se po ukončení požadavku

Pro organizaci HTTP autorizace:

- **username** - login
- **password** - heslo

7.3 jQuery - AJAX get () a sloupek () Methods

jQuery get () a post () metody jsou používány k žádosti o data ze serveru s HTTP GET nebo POST požadavku.

7.4 Požadavek HTTP: GET POST vs.

7.4.1 jQuery.get

Načítá stránku, používá proto GET požadavek. Může mít následující parametry:

- url požadované stránky
- odesílaná data (volitelný parametr)
- callback funkce, které bude odeslán výsledek (volitelný parametr)
- typ dat vracejících se do callback funkce (xml, html, script, json, text, _default)

7.4.2 jQuery.post

Tato metoda je obdobná te předchozí, jen odesílaná data budou odeslána prostřednictvím POST. Může mít následující parametry:

- url požadované stránky
- odesílaná data (volitelný parametr)
- callback funkce, které bude odeslán výsledek (volitelný parametr)
- typ dat vracejících se do callback funkce (xml, html, script, json, text, _default)

7.4.3 jQuery \$.get () Metoda

Metoda \$.get () požaduje data ze serveru s požadavkem HTTP GET.

Syntaxe:

`$.get(URL,callback);`

Povinný parametr URL specifikuje URL, kterou chcete požádat.

Volitelný parametr pro zpětné volání je název funkce, které mají být provedeny, pokud je požadavek úspěšný.

Následující příklad používá metodu \$.get () k načtení dat ze souboru na serveru:

Příklad

```
$("button").click(function(){
    $.get("demo_test.php", function(data, status){
        alert("Data: " + data + "\nStatus: " + status);
    });
});
```

První parametr \$.get () je URL požadavku ("demo_test.asp").

Druhý parametr je funkce zpětného volání.

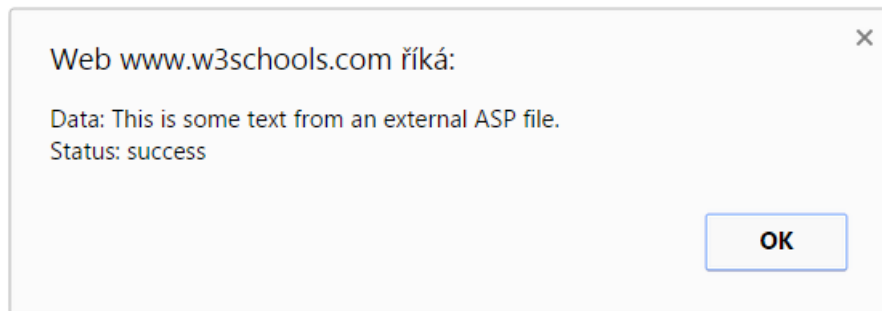
Parametr první zpětné volání obsahuje obsah požadované stránky, a druhý parametr zpětné volání obsahuje stav žádosti.

Tip: Zde je, jak se soubor ASP vypadá ("demo_test.asp"):

```
<%
response.write("This is some text from an external ASP file.")
%>
```

https://www.w3schools.com/jquery/tryit.asp?filename=tryjquery_ajax_get

Send an HTTP GET request to a page and get the result back



```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.1.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $.get("demo_test.asp", function(data, status){
            alert("Data: " + data + "\nStatus: " + status);
        });
    });
});
</script>
</head>
<body>

<button>Send an HTTP GET request to a page and get the result back</button>

</body>
</html>
```

7.4.4 jQuery \$.post () Metoda

Metoda \$.post() požaduje data ze serveru pomocí HTTP POST požadavku.

Syntaxe:

`$.post(URL,data,callback);`

Povinný parametr URL specifikuje URL, kterou chcete požádat.

Volitelný parametr dat určuje nějaká data k odeslání spolu s žádostí.

Volitelný parametr pro zpětné volání je název funkce, které mají být provedeny, pokud je požadavek úspěšný.

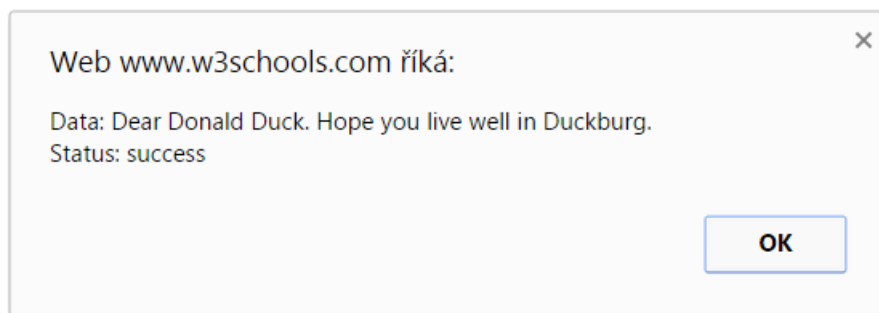
Následující příklad používá \$.post () metodu odesílat data spolu s žádostí:

Příklad

```
$("#button").click(function(){
    $.post("demo_test_post.asp",
    {
        name: "Donald Duck",
        city: "Duckburg"
    },
    function(data, status){
        alert("Data: " + data + "\nStatus: " + status);
    });
});
```

https://www.w3schools.com/jquery/tryit.asp?filename=tryjquery_ajax_post

Send an HTTP POST request to a page and get the result back



```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.1.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
    $("#button").click(function(){
        $.post("demo_test_post.asp",
        {
            name: "Donald Duck",
            city: "Duckburg"
        },
        function(data,status){
            alert("Data: " + data + "\nStatus: " + status);
        });
    });
});
</script>
```

```
</head>
<body>
<button>Send an HTTP POST request to a page and get the result back</button>
</body>
</html>
```

První parametr \$.post () je URL chceme požádat ("demo_test_post.asp").

Pak jsme se projít v některých údajů, které mají poslat spolu s žádostí (jméno a město).

Skript ASP v "demo_test_post.asp" přečte parametry, zpracovává je a vrátí výsledek.

Třetí parametr je funkce zpětného volání. Parametr první zpětné volání obsahuje obsah požadované stránky, a druhý parametr zpětné volání obsahuje stav žádosti.

Tip: Zde je, jak se soubor ASP vypadá ("demo_test_post.asp"):

```
<%
dim fname,city
fname=Request.Form("name")
city=Request.Form("city")
Response.Write("Dear " & fname & ". ")
Response.Write("Hope you live well in " & city & ".")
%>
```

7.5 jQuery.getJSON

Načítá data ve formátu JSON (obdobný formát jako XML). Může mít tyto parametry:

- url požadované stránky
- odesílaná data (volitelný parametr)
- callback funkce, které bude odeslán výsledek (volitelný parametr)

Příklad:

```
$(document).ready(function(){ // když je stránka načtena
    $('#example-4').click(function(){ // kliknutím na prvek s id = example-4
        $.getJSON('ajax/example.json', {}, function(json){ // se načte JSON ze souboru example.json
            $('#example-4').html("");
            // zaplníme DOM prvek daty z XML
            $('#example-4').append("To: " + json.note.to + '<br/>')
                .append('From: ' + json.note.from + '<br/>')
                .append('<b>' + json.note.heading + '</b><br/>')
                .append(json.note.body + '<br/>');
        });
    });
});
```

example.json

```
{
  note:{
    to:'Tove',
    from:'Jani',
    heading:'Reminder',
    body:'Don\'t forget me this weekend!'
  }
}
```

Ukázka: http://www.koding.cz/clanky/15/example_3.html

7.6 jQuery.getScript

Tato funkce načítá a provádí lokální JavaScript. Může mít tyto parametry:

- url požadované stránky
- callback funkce, které bude odeslán výsledek (volitelný parametr)

Příklad:

```
$(document).ready(function(){ // když je stránka načtena
    $('#example-5').click(function(){ // kliknutím na prvek s id = example-5
        $.getScript('ajax/example.js', function(){ // načteme JavaScript ze souboru example.js
            testAjax(); // provádíme načtený JavaScript
        });
    });
});
```

```
});
```

Soubor example.js

```
function testAjax() {  
    $('#example-5').html('Test completed'); //měníme prvek s id = example-5  
}
```

Ukázka

http://www.koding.cz/clanky/15/example_4.html

7.7 Odesílání formuláře

Pro odesílání formuláře prostřednictvím jQuery je vhodná jakákoliv z uvedených variant, ale pro pohodlné *sbírání* dat z formulářů je lepší použít plugin [jQuery From](#) <http://malsup.com/jquery/form/> nebo metody

serialize: <http://api.jquery.com/serialize/>

serializeArray(): <http://api.jquery.com/serializeArray/>

7.8 Odesílání souborů

Pro odesílání souborů prostřednictvím jQuery můžete použít plugin [Ajax File Upload](#).

<http://www.phpletter.com/Our-Projects/AjaxFileUpload/>

7.9 Spolupráce s PHP

Pro organizaci práce s PHP použít knihovnu [jQuery-PHP](#).

<http://jquery.hohli.com/>

7.9.1 getScript

Další funkcí, kterou uvedu, je `getScript()`. Ta načte skript ze serveru a poté ho spustí.

```
$.getScript("cesta/k.souboru");
```

Opět můžeme použít callback funkci jako parametr.

```
$.getScript("cesta/k.souboru", function(){  
    alert("Skript byl načten a spuštěn!");  
});
```

Obě funkce byly zkrácenou verzí funkce `ajax()`, podobně jako tomu bylo u událostí `sbind()`.

```
$.ajax({  
    url: "cesta/k.souboru",  
    dataType: 'script',  
    success: function(){  
        alert("Skript byl načten a spuštěn!");  
    }  
});
```

// načtení a spuštění skriptu pomocí obecné funkce pro AJAX

7.9.2 Příklad načtení HTML souboru funkcí ajax(), podobně jako na začátku funkcí load().

```
$.ajax({  
  url: "cesta/k.souboru",  
  dataType: 'html',  
  success: function(){  
    // po dokončení...  
    alert("Skript byl načten a spuštěn!");  
  }  
});
```

Parametr dataType může nabývat hodnot html, text, xml, script, json a jsonp. Pokud nepotřebujete nějakou speciální akci, vystačíte si se zkrácenými verzemi této funkce.

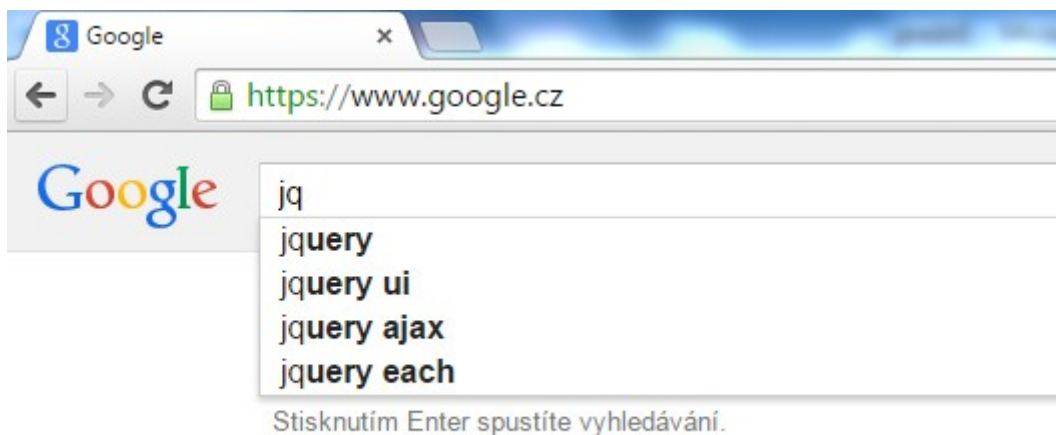
8 Funkce automatického dokončování

- Když uživatel zadává do formulářového pole nějaký obecně známý údaj nebo informaci, kterou už zadal někdy předtím (nebo někdo jiný), můžeme mu podstatně urychlit zadávání a současně mu naznačit, že zadává skutečně tu informaci, kterou po něm požadujeme.
- Jakmile uživatel vyplní do pole posloupnost znaků o určité délce, začneme mu zobrazovat možné hodnoty, které známe.
- Čím více znaků uživatel zadá, tím se bude seznam možných hodnot zmenšovat a bude přesnější.
- Tímto způsobem funguje funkce automatického dokončování, s níž se setkáváme v řadě desktopových aplikací a webových aplikací.

Příklad automatického dokončování na www.google.com

Na obrázku je vyhledávač Google s dvouznakovou posloupností jq ve vyhledávacím poli. Vyhledávač Google automaticky vyhledal nejpravděpodobnější vyhledávací fráze a na prvním místě nabídl slovo jquery a na druhém místě frázi jquery ui.

Tento vyhledávač aplikuje funkci automatického dokončování hned dvakrát - nejenomže nám poskytne seznam možných vyhledávacích frází, ale rovnou nejpravděpodobnější frázi vyhledá a zobrazí výsledky.



Abychom mohli tuto funkci implementovat, **musíme mít zdroj dat**, která budeme uživateli nabízet.

Zdrojem dat může být:

- pole hodnot v jazyce JavaScript – lokální zdroj
 - databáze na straně serveru - vzdálený zdroj dat.
-
- Například je jasné, že webové vyhledávače neuchovávají všechny vyhledávací fráze ve webovém prohlížeči klienta, ale ukládají je do své databáze na straně serveru.
 - Externí zdroje dat nám umožní ukládat podstatně větší množství hodnot.

9 Ovládací prvek Autocomplete

- součást knihovny jQuery UI
- umožňuje připojovat k formulářovým polím funkci automatického dokončování.
- Během toho, co uživatel píše údaj do pole, mu ovládací prvek **Autocomplete** zobrazuje návrhy možných hodnot. Tyto hodnoty můžeme načítat jak z lokálního zdroje, tak ze vzdáleného zdroje.

Zde je vše, včetně příkladů: <https://jqueryui.com/autocomplete/>

Api dokumentace je zde: <http://api.jqueryui.com/autocomplete/>

Syntaxe je:

```
$( "kam vložit" ).autocomplete({  
    source: zdroj  
});
```

Zdroje mohou být

Array

- An array of strings: ["Choice1", "Choice2"]
- An array of objects with label and value properties: [{ label: "Choice1", value: "value1" }, ...]

String

Autocomplete plugin očekává, že řetězec, který má poukázat na zdroj URL, vrátí data, JSON

```
$( "#skills" ).autocomplete({  
    source: 'search.php'  
});
```

Funkce

Function(Object request, Function response(Object data))

Při filtrování dat na místě, můžete využít

vestavěnou \$.ui.autocomplete.escapeRegexp funkci. Bude to trvat jeden argument řetězce a uniknout všem regex znaky, což je výsledek bezpečně předat new RegExp().

Například

```
<script>
$( "#autocomplete" ).autocomplete({
  source: [ "c++", "java", "php", "coldfusion", "javascript", "asp", "ruby"
]
});
</script>
```

Nebo

```
<script>
var tags = [ "c++", "java", "php", "coldfusion", "javascript", "asp",
"ruby" ];
$( "#autocomplete" ).autocomplete({
  source: function( request, response ) {
    var matcher = new RegExp( "^" +
$.ui.autocomplete.escapeRegex( request.term ), "i" );
    response( $.grep( tags, function( item ) {
      return matcher.test( item );
    }) );
  }
});
</script>
```

Případně z php

```
<script>
$(function() {
  $( "#skills" ).autocomplete({
    source: 'search.php'
  });
});
</script>
```

Dále je možné nastavit u autocomplete další vlastnosti:

Zpoždění

```
$( ".selector" ).autocomplete({  
  delay: 500  
});
```

nebo

```
// Getter  
var delay = $( ".selector" ).autocomplete( "option", "delay" );  
  
// Setter  
$( ".selector" ).autocomplete( "option", "delay", 500 );
```

Minimální délka pro rozpoznání

```
$( ".selector" ).autocomplete({  
  minLength: 0  
});
```

lépe

```
// Getter  
var minLength = $( ".selector" ).autocomplete( "option", "minLength" );  
  
// Setter  
$( ".selector" ).autocomplete( "option", "minLength", 0 );
```

9.1.1 Vytvoření ovládacího prvku Autocomplete

Ovládací prvek vytvoříme tak, že vybereme požadovaný element a zavoláme na něj metodu `autocomplete()`.

Například takto:

```
$('#element-s-automaticym-dokoncovanim').autocomplete();
```

```
<!doctype html>  
<html lang="en">  
<head>  
  <meta charset="utf-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1">  
  <title>jQuery UI Autocomplete - Default functionality</title>  
  <link rel="stylesheet" href="//code.jquery.com/ui/1.12.1/themes/base/jquery-  
ui.css">  
  <link rel="stylesheet" href="/resources/demos/style.css">  
  <script src="https://code.jquery.com/jquery-1.12.4.js"></script>  
  <script src="https://code.jquery.com/ui/1.12.1/jquery-ui.js"></script>  
  
<script>  
  $( function() {
```

```
var availableTags = [
  "ActionScript",
  "AppleScript",
  "Asp",
  "BASIC",
  "C",
  "C++",
  "Clojure",
  "COBOL",
  "ColdFusion",
  "Erlang",
  "Fortran",
  "Groovy",
  "Haskell",
  "Java",
  "JavaScript",
  "Lisp",
  "Perl",
  "PHP",
  "Python",
  "Ruby",
  "Scala",
  "Scheme"
];
$( "#tags" ).autocomplete({
  source: availableTags
});
</script>
</head>
<body>

<div class="ui-widget">
  <label for="tags">Tags: </label>
  <input id="tags">
</div>

</body>
</html>
```


10 Autocomplete textbox požití jQuery, PHP and MySQL

Musíme vložit knihovny:

<https://www.codexworld.com/autocomplete-textbox-using-jquery-php-mysql/>

<https://alpha.kts.vspj.cz/~smrcka/wt2/ajax/dokonci2/>

Nejprve se vytvoří tabulka

```
CREATE TABLE `skills` (  
  `id` int(11) NOT NULL AUTO_INCREMENT,  
  `skill` varchar(100) COLLATE utf8_unicode_ci NOT NULL,  
  `status` tinyint(1) NOT NULL DEFAULT '1' COMMENT '1=Active | 0=Inactive',  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci;
```

Nyní v index.html se načtou knihovny

```
<!-- jQuery library -->  
<script src = "https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js" ></script >  
  
<!-- knihovna uživatelského rozhraní jQuery -->  
<link rel = "stylesheet" href =  
"https://ajax.googleapis.com/ajax/libs/jqueryui/1.12.1/themes/smoothness/jquery-ui.css" >  
<script src = "https://ajax.googleapis.com/ajax/libs/jqueryui/1.12.1/jquery-ui.min.js" ></script >
```

Definování funkce automatického dokončování

Inicializace Autocomplete: funkce automatického doplňování inicializovat plugin.

`autocomplete()`

- Určete ID / třídu selektoru (`#skill_input`) vstupního pole, do které bude připojena funkce automatického doplňování.

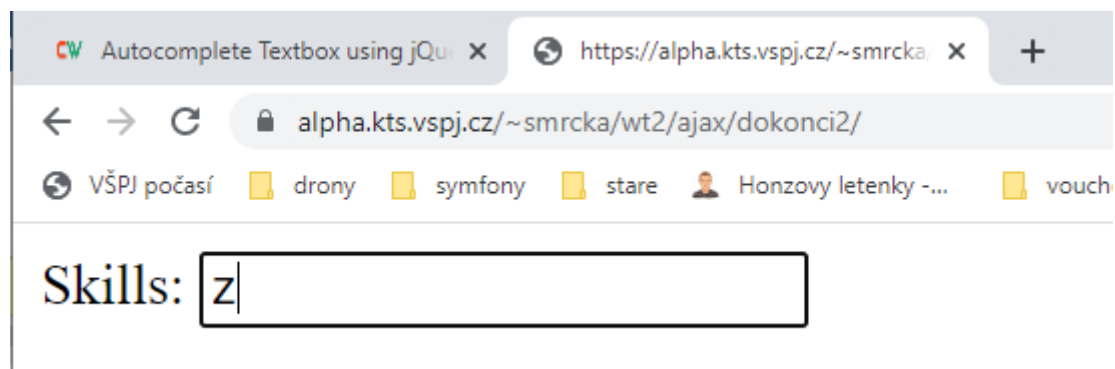
- Určete místní nebo vzdálený zdroj (`search.php`) pro načtení dat pro automatické návrhy.

```
< skript >
$ ( funkce () {
    $ ( "#skill_input" ) .autocomplete ({
        zdroj: "search.php" ,
    });
});
</ skript >
```

HTML

Potřebujeme následující HTML.

```
<label>Skills:</label>
<input type="text" id="skill_input"/>
```



PHP Zdrojový soubor

Hodnotu do textové pole získáme pomocí `term` pole (`$_GET['term']`) z řetězce dotazu. Nyní budeme načítat data z tabulky dovedností a filtrování dovedností `$_GET['term']` . Nakonec se vrátíme filtrovaná data na kvalifikaci ve formátu **JSON**.

Search.php

```
<?php

// Database configuration

$dbHost    = "localhost";

$dbUsername = "smrcka";

$dbPassword = "heslo";

$dbName    = "smrcka";


// Create database connection

$db = new mysqli($dbHost, $dbUsername, $dbPassword, $dbName);


// Check connection

if ($db->connect_error) {

    die("Connection failed: " . $db->connect_error);

}

// Get search term

$searchTerm = $_GET['term'];

// Fetch matched data from the database

$query = $db->query("SELECT * FROM skills WHERE skill LIKE '%".$searchTerm."%' AND
status = 1 ORDER BY skill ASC");


// Generate array with skills data

$skillData = array();

if($query->num_rows > 0){

    while($row = $query->fetch_assoc()){

        $data['id'] = $row['id'];

        $data['value'] = $row['skill'];

        array_push($skillData, $data);

    }

}

// Return results as json encoded array
```

```
echo json_encode($skillData);
```

```
?>
```

V případě, že chcete zobrazit i id, tak takto:

```
$(function() {  
    $("#skill_input").autocomplete({  
        source: "search.php",  
        select: function( event, ui ) {  
            event.preventDefault();  
            $("#skill_input").val(ui.item.id);  
        }  
    });  
});
```

minLength - Volitelné (výchozí 1). Minimální počet znaků, které musí být zadány před provedením vyhledávání.

```
$ (".selector ") .autocomplete ({  
    minLength: 5  
});
```


11 Vzdálený zdroj pro funkci automatického dokončování

<http://195.113.207.171/~smrcka/jquery/kniha/Kapitola05/>

Zdrojové kódy lze stáhnout na adrese:

<http://knihy.cpress.cz/ajax-d2.html>

The screenshot shows a web browser window with the URL <http://195.113.207.171/~smrcka/jquery/kniha/Kapitola05/>. The page has a dark header with the text "Rezervace školení". Below the header is a calendar for December 2011. The calendar shows days from 1 to 31, with some days highlighted in blue and green. Below the calendar is a form titled "Výběr školení a termínu". The form has a dropdown menu for "Školení:" with the selected value "Kaskádové styly (jazyk CSS)". Below this is a text area for "Popis školení:" containing the text "Na tomto školení se seznámíte s jazykem CSS. Školení pořádá zkušený kódér Jan Novák každé úterý od 16. do 17. hodiny. Cena je 300 Kč za hodinu." Below this is a text input field for "Termín:". Below the form is a section titled "Kontaktní údaje" with several input fields: "Jméno:", "Příjmení:", "E-mail:", "Ulice a číslo domu:", "Obec:" (with a dropdown menu showing "Dihlba" and a green checkmark), and "PSČ:". Below this is a section titled "Dokončení rezervace" with a text area for "Poznámka:". At the bottom left is a button labeled "Rezervovat".

Vylepšení příkladu - nabízení obcí ze vzdáleného zdroje dat.

- Jelikož sami takovým zdrojem dat nedisponujeme, použijeme databázi služby **Mapy Google od společnosti Google**.
- Společnost Google umožňuje přistupovat k této databázi prostřednictvím veřejně dostupného rozhraní **Google Maps JavaScript API** ve verzi 3. Začneme jednoduše tak, že vložíme do dokumentu našeho příkladu skript tohoto rozhraní. Rozšíříme tedy kód pro vkládání skriptů v souboru *index.html*:

```
<script type="text/javascript" src="js/jquery.js"></script>
```

```
<script type="text/javascript" src="js/jquery-ui.js"></script>
```

```
<script type="text/javascript" src="js/jquery.ui.datepicker-cs.js"></script>
```

```
<script type="text/javascript" src="js/ajax.js"></script>
```

```
<script type="text/javascript"
```

```
src="http://maps.google.com/maps/api/js?sensor=false&language=cs">
```

```
</script>
```

```
<script type="text/javascript" src="js/hlavni.js"></script>
```

UPOZORNĚNÍ

Skript <http://maps.google.com/maps/api/js> je umístěný v Internetu, takže pro funkčnost tohoto příkladu, musíme být připojeni k Internetu.

V adrese URL k danému skriptu používáme rovněž dva parametry:

- Parametrem **sensor** sdělujeme tomuto rozhraní, zda se dotazujeme na data ze zařízení, které obsahuje lokalizační zařízení (například lokalizační jednotku GPS). Hodnota false označuje, že náš počítač lokalizační zařízení nemá.
- V parametru **language** nastavujeme kód jazyka pro dané rozhraní. Pro češtinu použijeme kód cs. Seznam ostatních jazyků je k dispozici kupříkladu na adrese <https://spreadsheets.google.com/pub?key=p9pdwsai2hDMsLkXsoM05KQ&gid=1>.

Jelikož jsme úspěšně vložili rozhraní Google Maps JavaScript API do své aplikace, můžeme začít používat jeho funkce. Konkrétně použijeme jedinou funkci, a to funkci `geocode()`, jež je metodou objektu typu `google.maps.Geocoder`. Vytvoříme si tedy nejprve tento objekt, a to tak, že přidáme níže uvedenou definici proměnné na začátek souboru *hlavni.js*:

```
var minimalniDatum = 0,  
    maximalniDatum = '+2m',  
    rezervovatelneDny,  
    geocoder = new google.maps.Geocoder();
```

Tímto novým řádkem jsme vytvořili objekt typu **google.maps.Geocoder** a uložili ho do globální proměnné **geocoder**.

Metoda geocode

- <https://developers.google.com/maps/documentation/javascript/geocoding#GeocodingResults>
- Tato metoda hledá geografická místa v databázi služby Mapy Google na základě zadaného výrazu.
- Nevyhledává nutně jen obce, ale také jejich části, významné budovy apod.
- Proto budeme nutné ještě dodatečně filtrovat obce ve výsledcích této metody.

První argument předáváme této metodě objekt požadavku s **vyhledávaným výrazem** a jako **druhý argument** funkci zpětného volání pro zpracování **odpovědi** s výsledky.

Nyní už můžeme téměř přejít k psaní kódu, ale měli bychom si ujasnit, kam vložit volání metody `geocode()`. Víme, že naše funkce `nahti NavrhyObci ()` v souboru *hlavni.js* má mimo jiné za úkol hledat obce podle zadaného výrazu, proč tedy nedelegovat toto vyhledávání na metodu `geocode()`? Přesně to uděláme - přepíšeme funkci `nahti NavrhyObci()` tímto způsobem:

```
/**  
 * Načte návrhy obcí pomocí funkce rozhraní Google Maps JavaScript API  
 * ve verzi 3.  
 * @param {Object} požadavek Objekt požadavku s vlastností term, která  
 * obsahuje aktuální hodnotu příslušného textového pole.  
 * @param {function} odpovez Funkce, s jejíž pomocí vrátíme návrhy obcí  
 * v podobě pole s objekty obsahujícími vlastnosti label a value.  
 */  
/**
```

```
function nactiNavrhyObci(pozadavek, odpovez) {
  geocoder.geocode(
    {
      address: pozadavek.term,
      region: 'CZ'
    },
    function(mista) {
      var pocetMist = mista.length,
          navrhy = [],
          misto,
          jeObec;
      for (var poradiMista = 0; poradiMista < pocetMist; poradiMista++) {
        misto = mista[poradiMista];
        jeObec = misto.types.indexOf('locality') != -1;
        if (jeObec) {
          navrhy.push({
            label: misto.formatted_address,
            value: misto.address_components[0].long_name
          });
        }
      }
      odpovez(navrhy);
    }
  );
}
```

Celé tělo této funkce se skládá pouze z výše uvedeného volání metody **geocode()** na náš objekt typu `google.maps.Geocoder`, který jsme si předtím uložili do globální proměnné `geocoder`.

Dané metodě předáváme dva argumenty.

```
{
  address: pozadavek.term,
  region: 'CZ' },
```

Toto je první z nich - objekt požadavku. Pomocí vlastnosti `address` specifikujeme vyhledávanou adresu, proto používáme text, který uživatel zadal do pole `Obec`.

Funkce `nactiNavrhyObci()` získáme text jako hodnotu vlastnosti požadavku `term`.

Pomocí vlastnosti `region` deiniujeme upřednostňovanou oblast, jejíž hodnota se nachází v seznamu na adrese <http://www.iana.org/assignments/language-subtag-registry>. Pro Českou republiku používáme hodnotu `'CZ'`.

Jelikož jsme službě Mapy Google řekli, co chceme hledat a v jaké oblasti především, vyhledá odpovídající geografická místa a vrátí nám odpověď. Odpověď vrací tak, že volá funkci zpětného volání (druhý argument metody `geocode()`) a předává jí pole s vyhledanými místy:

```
function(mista) {
  ...
}
```

Používáme anonymní funkci s parametrem `mista`, do nějž nám služba Mapy Google vloží pole s vyhledanými místy. V této funkci opět začínáme definicemi proměnných.

```
var pocetMist = mista.length,
```

Proměnná **pocetMist** je pomocná proměnná obsahující počet vyhledaných míst.

```
navrhry = [],
```

Definujeme pole návrhy, které nakonec předáme jako odpověď ovládacímu prvku Auto-complete. Tímto krokem opět zahajujeme transformaci dat do formátu, který vyžaduje ovládací prvek Autocomplete.

```
misto,
```

Proměnná **misto** je pomocná proměnná, do níž budeme ukládat aktuální místo v transformačním cyklu `for`.

```
jeObec;
```

Proměnná **jeObec** je další pomocnou proměnnou, do které si uložíme příznak, jestli je aktuální místo obcí.

```
for (var poradiMista = 0; poradiMista < pocetMist; poradiMista++) { misto =  
mista[poradiMista] ;
```

Stejně jako v minulé variantě naší funkce `navrhObci()` používáme cyklus `for`. Předtím však sloužil k vyhledávání a transformaci dat, ale tentokrát slouží k filtrování výsledků ze služby Mapy Google (na kterou jsme delegovali vyhledávání) a transformaci dat. Na začátku daného cyklu si ukládáme aktuální místo z pole `mista` do proměnné **misto**.

```
jeObec = misto.types.indexOf('locality') !== -1;
```

Vyhledané geografické místo je objektem. Jednou z vlastností tohoto objektu je vlastnost `types`, která představuje pole typů. Každé místo ve službě Mapy Google může mít totiž více typů. Nás zajímá jediná věc - jestli je mezi typy aktuálního místa i typ `'locality'`, který mají všechny obce. Pokud má aktuální místo typ `'locality'`, proměnná **jeObec** bude obsahovat hodnotu `true`; v opačném případě bude obsahovat hodnotu `false`.

```
if (jeObec) {
```

```
...
```

```
}
```

Předchozím příkazem `if` filtrujeme z výsledků pouze obce - ostatní geografická místa nás nezajímají.

```
navrhry.push({  
  label: misto.formatted_address,  
  value: misto.address_components[0].long_name });
```

Přidáváme aktuální obec do výsledků pro ovládací prvek Autocomplete, přičemž převádíme data do patřičného formátu.

Objekt **misto** má kromě vlastnosti **types** také vlastnost **formatted_address** a **address_components**. Hodnotou vlastnosti **formatted_address** je adresa dané obce přizpůsobená pro výpis uživateli (název obce, stát a někdy i poštovní směrovací číslo), proto ji nastavujeme jako popis položky v rozvíracím seznamu s návrhy (vlastnost `label`).

Hodnotou vlastnosti **address_components** je pole komponent adresy dané obce (každá komponenta je objektem). Hlavní komponentou obce je první komponenta (s indexovým

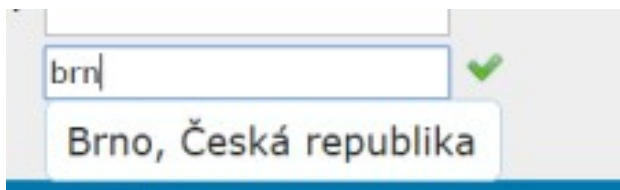
pořadím 0), jejíž vlastnost **long_name** obsahuje název obce (nikoliv už kompletní adresu), a ten nastavujeme jako hodnotu, která by se měla vložit do pole Obec (vlastnost value).

TIP

Pokud se chcete dozvědět více informací o rozhraní Google Maps JavaScript API verze 3, najdete je na adrese

<https://developers.google.com/maps/documentation/javascript/>

<https://developers.google.com/maps/documentation/javascript/examples/>



12 Vývoj a ladění ajaxových aplikací

12.1 Výpis informací z hlavičky

Základem ajaxových aplikací je přenos údajů přes XMLHttpRequest. Proto je důležité mít možnost vypsat údaje o hlavičkách protokolu http.

Informace o hlavičkách se nalezne:

<http://www.kosek.cz/clanky/iweb/05.html>

12.1.1 Nejpoužívanější hlavičky

Verze protokolu HTTP 1.0 definovala 17 hlaviček. HTTP/1.1 tento počet ještě zvětšilo. My si stručně objasníme význam nejdůležitějších hlaviček.

12.1.2 Content-Type

Tato hlavička udává typ přenášených dat. Typ dat se zapisuje pomocí MIME konvence. Pro HTML stránky máme typ text/html, pro obyčejný text text/plain, obrázky mají podle použitého formátu jeden z typů image/gif, image/jpeg nebo image/png. Podle typu dat prohlížeč pozná, jak přichází data interpretovat. HTML stránka zasílaná prohlížeči jako odpověď, proto mezi hlavičkami obsahuje následující řádku

Content-Type: text/html

12.1.3 Location

Tato hlavička obsahuje adresu dokumentu, který byl přesunut. Tuto hlavičku server posílá v případech, kdy stavový kód požadované operace začíná na 3. Prohlížeč většinou automaticky nahraje stránku, na kterou Location ukazuje. Jako adresu je potřeba uvést úplné absolutní URL. Např.

Location: http://manes.vse.cz/~xkosj06/index.html

12.1.4 If-Modified-Since

Pokud v požadavku použijeme tuto hlavičku společně s nějakým datem, server nám požadovaný objekt vrátí pouze, pokud byl od zadaného data změněn. Příklad: Chceme získat dokument pouze, pokud se od 30. března 1998 změnil:

If-Modified-Since: Mon, 30 Mar 1998 12:00:00 GMT

12.1.5 User-Agent, Server

V hlavičce User-Agent posílá prohlížeč svoji identifikaci -- obvykle své jméno, číslo verze a platformu, na které je spuštěn. Server naopak obsahuje identifikaci serveru, který vyřídil požadavek. Příklad:

User-Agent: Mozilla/4.0 (compatible; MSIE 4.01; Windows NT)

Server: Apache/1.3b3

Informace vypíšeme pomocí metody **xHttpRequest.getAllResponseHeaders()**.

```
<html>
<head>
<meta http-equiv="content-type" content="text/html;
charset=windows-1250">
<title>Informace z hlaviček</title>

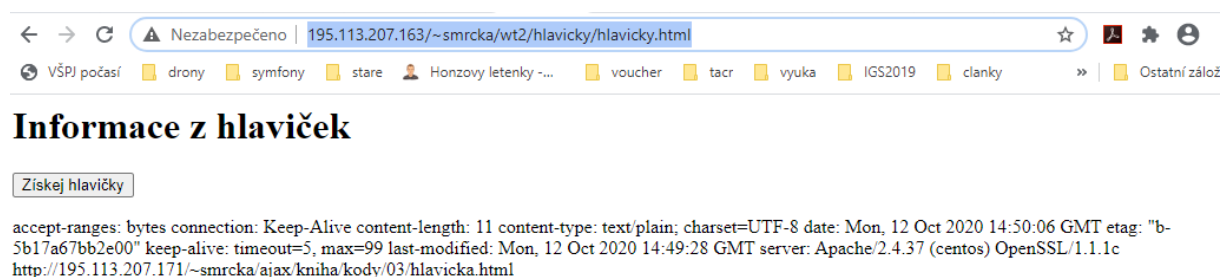
    <script language = "javascript">1

var xHttpRequest;
if (window.ActiveXObject)
    { xHttpRequest = new ActiveXObject("Microsoft.XMLHTTP");}
else if (window.XMLHttpRequest)
    { xHttpRequest = new XMLHttpRequest();}

function aktivujPozadavek(element)
{
    var obj = document.getElementById(element);
    xHttpRequest.open("HEAD", "text.txt");
    xHttpRequest.onreadystatechange = function()
    {
        if (xHttpRequest.readyState == 4 && xHttpRequest.status == 200)
            {obj.innerHTML = xHttpRequest.getAllResponseHeaders();}
    }
    xHttpRequest.send(null);
}
</script>

</head>
<body>
    <h1>Informace z hlaviček</h1>
    <form>
        <input type = "button" value = "Získej hlavičky"
            onclick = "aktivujPozadavek('oznam')">
    </form>
    <div id = "oznam"><div></div></div>
</body>
</html>
```

<http://195.113.207.163/~smrcka/wt2/hlavicky/hlavicky.html>

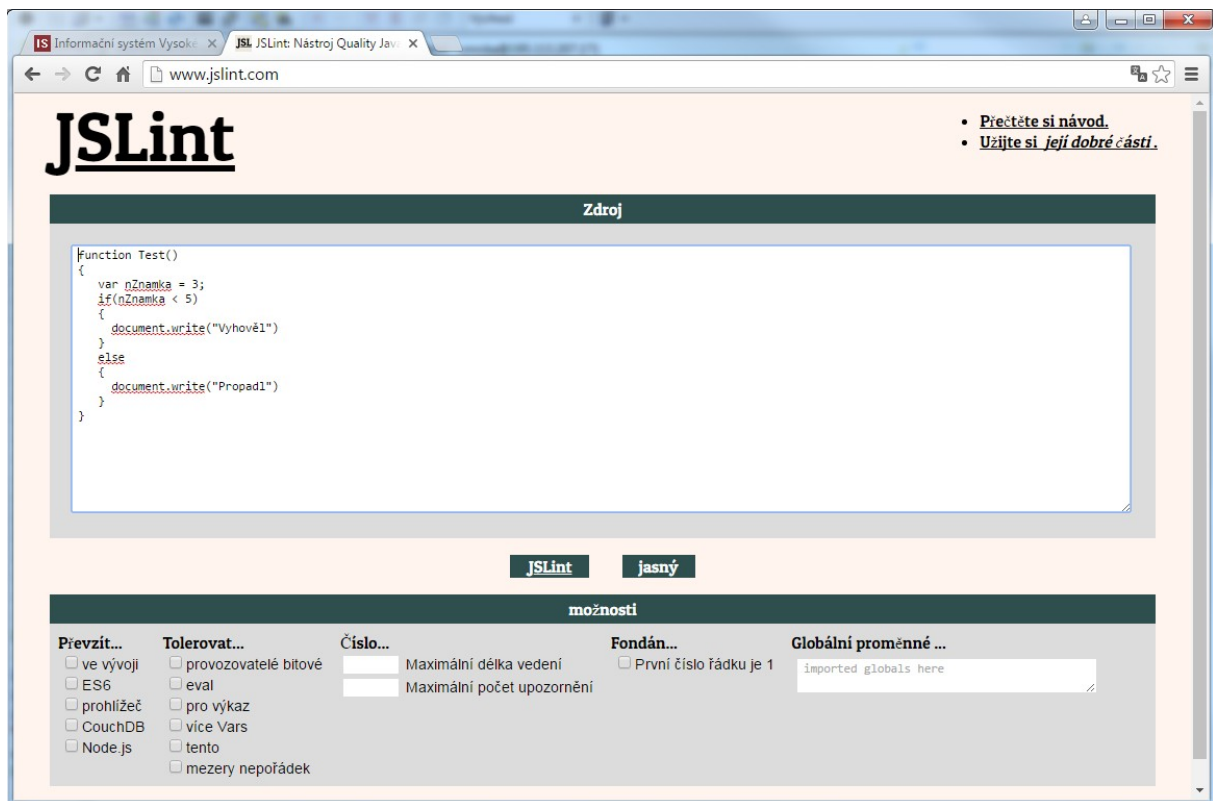


The screenshot shows a web browser window. The address bar displays the URL `http://195.113.207.163/~smrcka/wt2/hlavicky/hlavicky.html`. The browser's toolbar includes various icons for navigation and security. Below the address bar, there is a horizontal menu with links: `VŠPJ počasí`, `drony`, `symfony`, `stare`, `Honzovy letenky -...`, `voucher`, `tacr`, `vyuka`, `IGS2019`, `clanky`, and `Ostatní zálož`. The main content area of the browser shows the title **Informace z hlaviček** and a button labeled `Získej hlavičky`. Below the button, there is a block of text containing technical details about the connection, including headers like `accept-ranges`, `bytes connection`, `Keep-Alive`, `content-length`, `content-type`, `charset`, `date`, `etag`, `keep-alive`, `timeout`, `last-modified`, and `server`.

13 Ladění ajaxových aplikací

Využití nástroje JSLint

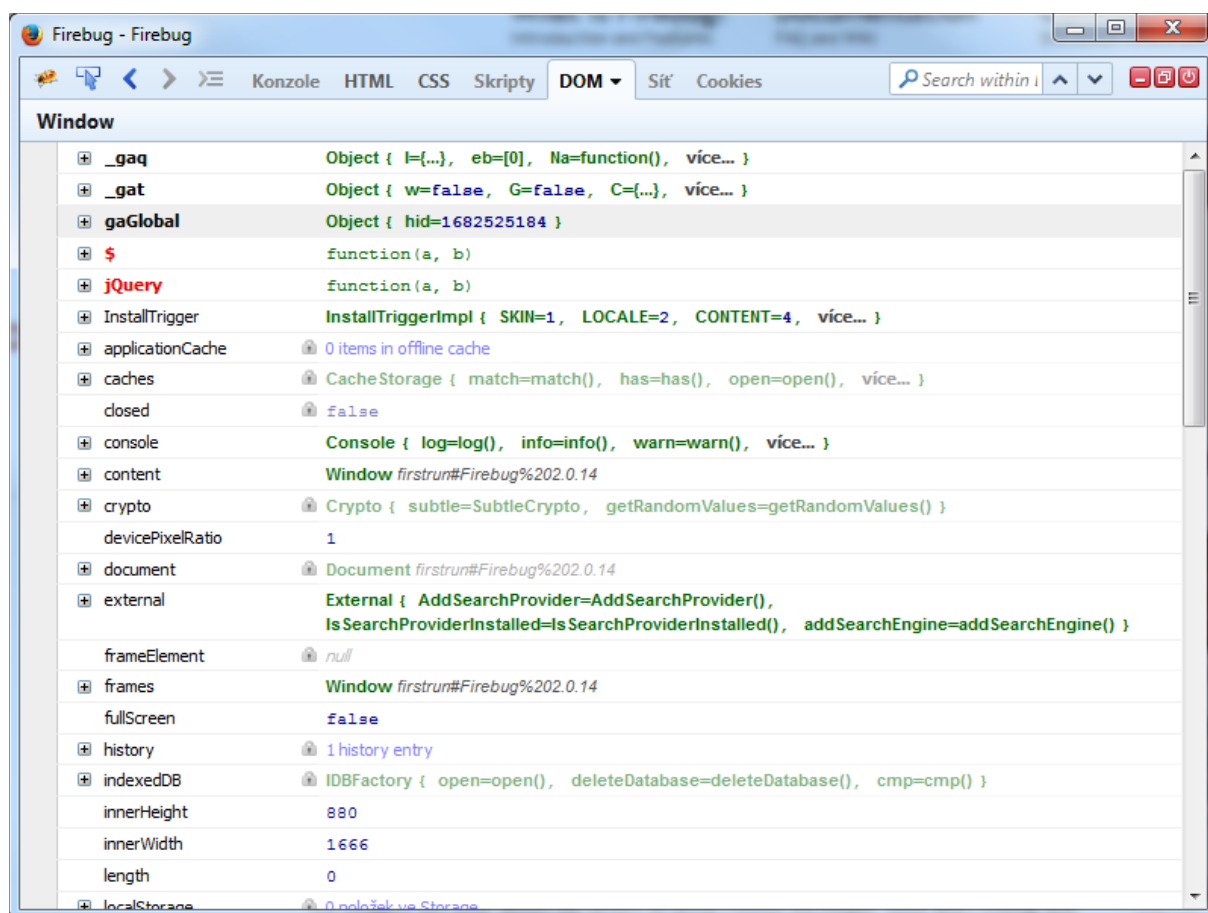
<http://www.jslint.com/>



13.1 Využití nástroje Microsoft Script Debugger

13.2 Nástroj Firebug

- Rozšíření pro Firefox
- Umožní kontrolovat a ladění kódu za webové stránky.
- Hledání chyby v jejich kódu
- Lze provádět živé změny HTML a CSS
- Hledat prvky webové stránky kodexu
- Ladit JavaScript
- Lze jej otevřít ve vlastním okně stiskem "Ctrl" a "F12"



Je možné tedy trasovat XMLHttpRequest požadavky pro Ajax

Je možné sledovat časové odezvy.

Základní verze je doplňku FireBug je dostupná pouze pro prohlížeč FireFox.

Pro ostatní prohlížeče je k dispozici FireBug Lite na adreses <https://getfirebug.com/firebuglite>

14 JSON : jednotný formát pro výměnu dat

Zdroj: <https://www.zdrojak.cz/clanky/json-jednotny-format-pro-vymenu-dat/>

JSON (JavaScript Object Notation) se v posledních letech stal jedním z nejpoužívanějších formátů pro výměnu dat na Webu.

JSON se objevil ve chvíli, kdy se na webu pro výměnu dat používal převážně formát XML. Ten v očích javascriptových vývojářů trpěl některými nedostatky, např. práce s ním byla složitá (bylo nutné používat „neohrabaný“ DOM, řešit přítomnost uzlů obsahujících pouze bílé znaky apod.).

A tak ačkoliv při zápisu celých dokumentů JSON nemůže (a ani nemá) formátu XML konkurovat, v zápisu krátkých strukturovaných dat vyměňovaných webovými aplikacemi konkurenční boj vyhrál JSON.

Zápis JSON je platným zápisem jazyka JavaScript. To je jedna z jeho výhod; již ze samotného pohledu na zápis v JSON rovnou vidíme, jak s ním budeme v programu pracovat:

```
1. [
2.  {"name": "Cerna sanitka", "tvname": "CT1"},
3.  {"name": "Comeback", "tvname": "Nova"},
4.  {"name": "Pratele", "tvname": "Prima"}
5. ]
```

Pokud by výše uvedená data nebyla v JSON, ale v čistém textu, navrhli bychom si datovou strukturu, do které bychom je načetli (byla by pravděpodobně stejná nebo velmi podobná našemu zápisu v JSON). Pokud by data byla v XML, postupovali bychom stejně nebo bychom je zpracovali skrze rozhraní DOM. Oba způsoby jsou možné, ale přesně o jeden krok komplikovanější, než je nezbytně nutné.

Zápis v JSON je zápisem v JavaScriptu, je tedy sám o sobě již připravenou datovou strukturou ke zpracování. Můžeme říct, že JSON je serializovanou podobou datových typů JavaScriptu. Přesněji některých jeho datových typů, viz dále.

14.1.1 Formát JSON

Do JSON můžeme uložit následující typy dat:

- JSONString – textový řetězec
- JSONNumber – číslo (celočíslné nebo reálné, včetně zápisu s exponentem)
- JSONBoolean – logická hodnota
- JSONNull – hodnota null
- JSONArray – pole
- JSONObject- objekt

Ostatní datové typy nemůžeme vkládat přímo, např. datum pro vložení do JSON převedeme na JSONString.

14.1.2 JSONString

Příklad použití: "Hezky česky"

Řetězec musí být vložen do uvozovek (apostrofy nejsou povoleny) a může obsahovat všechny znaky Unicode. Uvozovky a zpětné lomítko je nutné vložit ve tvaru " a \.

Pokud bychom měli s češtinou problémy, můžeme znaky s diakritikou stejně jako v JavaScriptu vložit ve tvaru uXXXX, kde XXXX značí kód znaku z tabulky Unicode zapsaný v šestnáctkové soustavě. V našem případě by výsledek vypadal: "Hezky u010desky".

14.1.3 JSONNumber

Příklady:

12
-13.5
0.7e-4

Na zápisu čísla není nic neobvyklého, snad jen, že vedoucí nulu před desetinnou čárkou nelze vynechat. Zápis .5 je tedy neplatný.

14.1.4 JSONBoolean

Příklady:

true
false

14.1.5 JSONNull

Příklad: null

14.1.6 JSONArray

Příklady:

[0, "Shakespeare", false]

[[1, 2], null, "apple"]

Pole v JSON je kontejner obsahující seřazený výpis hodnot. Ohraničují jej hranaté závorky. Hodnotami pole může být kterýkoliv datový typ JSON včetně objektu nebo pole; můžeme tak pole do sebe snadno vnořovat.

14.1.7 JSONObject

Příklady:

{"x": 45, "y": 78}

{"person": {"name": "Robin", "age": 35}, "met": false, "hobbies": null, "data": [1, 2]}

Objekt v JSON není plnohodnotný objekt, jaký známe z JavaScriptu. Jedná se o kontejner, který obsahuje pouze data, žádné metody (měli bychom ho tedy správně nazývat hash nebo asociativní pole). Každá datová položka objektu má svůj klíč, který je typu JSONString.

Do objektu můžeme vkládat další objekty; lze tak snadno vytvářet složitější struktury. Rozsáhlost struktur a hloubka vnoření není specifikací omezena, může být ovšem omezena konkrétní implementací JavaScriptu a JSON parseru.

Tím jsme si předvedli stavební kameny JSON. Nyní můžete začít s tvorbou datových formátů.

14.1.8 Nástroje JSONLint a JSON Editor

Při práci s JSON vám přijdou vhod některé nástroje.

Tím prvním je JSONLint na adrese jsonlint.com, který váš JSON zápis zvaliduje a pomůže vám opravit chyby v syntaxi. Zároveň také všechna data jednotně odsadí (není nutné, ale zvyšuje to čitelnost).

JSONLint

The JSON Validator

An Arc90 Labs Production

Props to Douglas Crockford of JSON and JS Lint, Ben Spencer of jsonval, The Blueprint CSS Framework, and jQuery

```
[
  {
    "name": "Černa sanitka",
    "tvname": "CT1"
  },
  {
    "name": "Comeback",
    "tvname": "Nova"
  },
  {
    "name": "Pratele",
    "tvname": "Prima"
  }
]
```

Validate

Want to make nifty tools like this during work hours?
[Check Us Out and Say Hello.](#)

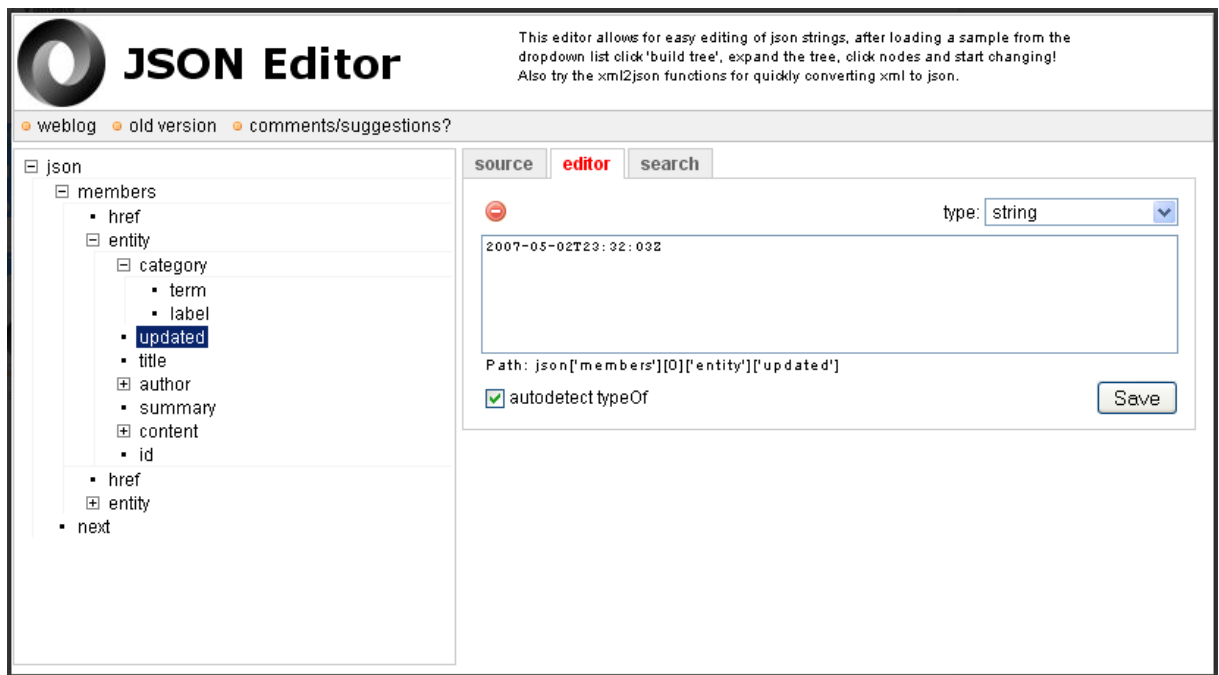
[FAQ](#)

Results

syntax error, unexpected '}', expecting ']' at line 9
Parsing failed

Dalším nástrojem je [JSON Editor](http://braincast.nl/samples/jsoneditor/) <http://braincast.nl/samples/jsoneditor/>. V něm si můžete vyzkoušet práci se složitějšími strukturami. Zobrazí vám nejen jejich stromovou podobu, ale umožní v ní i vyhledávat a upravovat jednotlivé uzly. U složitých struktur oceníte možnost vygenerování kódu pro přístup k uzlu. Stačí vybrat požadovaný uzel a editor zobrazí kód pro přístup k němu, např.:

1. `json['members'][0]['entity']['updated']`



14.2 Použití externí adresy

Přestože je JSON odvozen z JavaScriptu, je jazykově nezávislý. Najdeme tak implementaci JSON v [PHP](#) (od verze 5.2 nativně), [Ruby](#) nebo [Pythonu](#), a dokonce některé desktopové aplikace používají JSON jako souborový formát (např. prohlížeč [Google Chrome](#)).

Na Webu se JSON používá pro výměnu dat pomocí AJAXu.

Konkuruje mu zde asi jen XML. Navíc řada služeb používá JSON pro výměnu dat skrze své API.

Externí URL

Ajax jQuery –

hat Bigood said is true.. you are getting confused between POST and GET.

To use GET use below code:

```
$.ajax({
  url: '/path/addId?type=one&id=' + $("#id").val(),
  type: "GET",
  success: function(data){
    alert(data[message]);
  },
  error: function(data){
    alert("error!");
  }
})
```

```
});
```

to use POST use below code

```
var id=$("#id").val();
$.ajax({
  url: '/path/addId',
  type: "POST",
  data: {type:'one',id:id},
  success: function(data){
    alert(data[message]);
  },
  error: function(data){
    alert("error!");
  }
});// indentation
```